

Ionian University

Department of Informatics



Undergraduate Thesis

Developing a Mobile-Assisted Language Learning Application Utilizing Short-Form Video Reels and Hypercasual Games

Mohammad Matin Marzie

Registration Number: inf2022001

Supervisor: Prof. Nikolaos Korfiatis

September 21, 2026

Ιόνιο Πανεπιστήμιο

Τμήμα Πληροφορικής



Πτυχιακή Εργασία

**Ανάπτυξη Εφαρμογής για Φορητές Συσκευές για τη
Γλωσσική Εκμάθηση αξιοποιώντας Βίντεο Μικρού
Μήκους και Hypercasual Παιχνιδιών**

Mohammad Matin Marzie

Αριθμός Μητρώου: inf2022001

Επιβλέπων: Καθ. Νικόλαος Κορφιάτης

21 Σεπτεμβρίου 2026

Supervisor

Nikolaos Korfiatis, *Professor,*
Ionian University

Examination Committee

Nikolaos Korfiatis, *Professor,*
Ionian University

Katia-Lida Kermanidou, *Professor,*
Ionian University

Ioannis Karydis, *Associate Professor,*
Ionian University

I dedicate this work to the 168 schoolgirls and educators of the Shajareh Tayyebah School in Minab, who lost their lives in the tragic school bombing of February 2026. May we move forward to build a world where schools remain safe sanctuaries for every child

All copyrights are reserved by the author.

Abstract

Mobile-assisted language learning (MALL) applications are now the most widespread medium of independent language study, yet most of them rest on uniform, decontextualised drills decorated with points and streaks, and first-week attrition is reported to exceed 60%. This thesis asks whether the media that learners already consume habitually — short-form video and hypercasual games — can be turned into an effective, personalised vocabulary-learning environment. It presents *Glosy*, a cross-platform mobile application that currently supports English, Greek, and Farsi in any native/learning combination, with a dictionary of roughly 41,000 words tagged by proficiency level. The central design element is a word-level learner model: every word a learner tracks carries a Free Spaced Repetition Scheduler (FSRS) memory state (difficulty, stability, and retrievability) that is updated on the device after each graded review, so the application works offline and synchronises when a connection returns. Learners watch reels — videos of at most 90 seconds with timestamped, tokenised, translated subtitles, which users can also create — and add words to their vocabulary by tapping them. A three-stage cascade recommender selects the reels. A comprehensibility filter keeps only reels of which the learner already knows at least 98% of the vocabulary; a spaced-repetition stage ranks reels containing words that are due for review first, so that watching a reel doubles as a graded review; and a content-based stage orders the remainder by the TF-IDF cosine similarity of each reel to a profile built from the learner’s engagement. Two hypercasual games, a multilingual Wordle-style game and a procedurally generated crossword called Word of Wonders, draw their words from the same tracked vocabulary, so that each round is also a spaced-repetition review.

No controlled study with learners has yet been conducted, so pedagogical effectiveness is not claimed; instead, the algorithmic components are evaluated directly. Over 4,200 benchmark trials the crossword generator never failed, and the richness of its boards was limited by its letter budget rather than its word cap. *Glosy* also faces the cold-start problem of a new application: with no user base yet, no users have uploaded reels, so its content library is small and the recommender

has no interaction history to personalise from. The thesis closes with the main limitations — the absence of a user study and offline annotation of new reels — and with directions for future work: automated annotation, cold-start mitigation, and a randomised controlled evaluation.

Key words: mobile-assisted language learning, short-form video, spaced repetition, recommender systems, hypercasual games, vocabulary acquisition

Περίληψη

Οι εφαρμογές εκμάθησης γλωσσών με φορητές συσκευές (Mobile-Assisted Language Learning, MALL) αποτελούν σήμερα το πιο διαδεδομένο μέσο αυτόνομης γλωσσικής μάθησης, ωστόσο οι περισσότερες στηρίζονται σε ομοιόμορφες ασκήσεις αποκομμένες από το γλωσσικό πλαίσιο, «διακοσμημένες» με πόντους και σερί ημερών, ενώ η απώλεια χρηστών μέσα στην πρώτη εβδομάδα αναφέρεται ότι ξεπερνά το 60%. Η παρούσα εργασία εξετάζει αν τα μέσα που οι μαθητές ήδη καταναλώνουν συστηματικά — τα βίντεο μικρού μήκους και τα παιχνίδια hypercasual — μπορούν να μετατραπούν σε ένα αποτελεσματικό, εξατομικευμένο περιβάλλον εκμάθησης λεξιλογίου. Παρουσιάζει το *Glosy*, μια εφαρμογή για φορητές συσκευές πολλαπλών πλατφορμών (cross-platform), η οποία υποστηρίζει προς το παρόν Αγγλικά, Ελληνικά και Φαρσί σε οποιονδήποτε συνδυασμό μητρικής και εκμαθούμενης γλώσσας, με λεξικό περίπου 41.000 λέξεων επισημασμένων ανά επίπεδο επάρκειας.

Κεντρικό στοιχείο του σχεδιασμού είναι ένα μοντέλο μαθητή σε επίπεδο λέξης: κάθε λέξη που παρακολουθεί ο μαθητής φέρει μια κατάσταση μνήμης του Free Spaced Repetition Scheduler (FSRS) (δυσκολία, σταθερότητα και ανακτησιμότητα), η οποία ενημερώνεται στη συσκευή μετά από κάθε βαθμολογημένη επανάληψη, ώστε η εφαρμογή να λειτουργεί χωρίς σύνδεση και να συγχρονίζεται όταν αυτή επανέλθει. Οι μαθητές παρακολουθούν reels — βίντεο διάρκειας έως 90 δευτερολέπτων με χρονισμένους, χωρισμένους σε λέξεις και μεταφρασμένους υπότιτλους, τα οποία μπορούν και οι ίδιοι να δημιουργήσουν — και προσθέτουν λέξεις στο λεξιλόγιό τους πατώντας πάνω τους. Τα reels επιλέγονται από ένα σύστημα συστάσεων τριών σταδίων τύπου «καταρράκτη» (cascade). Ένα φίλτρο κατανοησιμότητας κρατά μόνο τα reels στα οποία ο μαθητής γνωρίζει ήδη τουλάχιστον το 98% του λεξιλογίου· ένα στάδιο επανάληψης σε αραιά διαστήματα προτάσσει τα reels που περιέχουν λέξεις προς επανάληψη, ώστε η παρακολούθηση ενός reel να λειτουργεί ταυτόχρονα ως βαθμολογημένη επανάληψη· και ένα στάδιο βασισμένο στο περιεχόμενο ταξινομεί τα υπόλοιπα με βάση την ομοιότητα συνημιτόνου (TF-IDF) κάθε reel με ένα προφίλ που δημιουργείται από τις αλληλεπιδράσεις του μαθητή. Δύο παιχνίδια hypercasual, ένα πολύγλωσσο παιχνίδι τύπου Wordle

και ένα διαδικαστικά παραγόμενο σταυρόλεξο με την ονομασία Word of Wonders, αντλούν τις λέξεις τους από το ίδιο λεξιλόγιο που παρακολουθεί ο μαθητής, ώστε κάθε γύρος να αποτελεί επίσης επανάληψη σε αραιά διαστήματα.

Δεν έχει ακόμη πραγματοποιηθεί ελεγχόμενη μελέτη με μαθητές, και για τον λόγο αυτό δεν υποστηρίζεται η παιδαγωγική αποτελεσματικότητα· αντί γι' αυτό, αξιολογούνται απευθείας τα αλγοριθμικά συστατικά του συστήματος. Σε 4.200 δοκιμές αξιολόγησης (benchmark) ο γεννήτορας του σταυρολέξου δεν απέτυχε ποτέ, και ο πλούτος των πλεγμάτων του περιοριζόταν από το όριο γραμμάτων και όχι από το όριο λέξεων. Το *Glosy* αντιμετωπίζει επίσης το πρόβλημα ψυχρής εκκίνησης (cold start) μιας νέας εφαρμογής: χωρίς ακόμη βάση χρηστών, κανένας χρήστης δεν έχει ανεβάσει reels, οπότε η βιβλιοθήκη περιεχομένου είναι μικρή και η μηχανή συστάσεων δεν διαθέτει ιστορικό αλληλεπιδράσεων για εξατομίκευση. Η εργασία κλείνει με τους βασικούς περιορισμούς — την απουσία μελέτης με χρήστες και την εκτός σύνδεσης επισημείωση των νέων reels — και με κατευθύνσεις για μελλοντική εργασία: αυτοματοποιημένη επισημείωση, μετριασμό της ψυχρής εκκίνησης και τυχαιοποιημένη ελεγχόμενη αξιολόγηση.

Λέξεις κλειδιά: εκμάθηση γλωσσών με φορητές συσκευές, βίντεο μικρού μήκους, επανάληψη σε αραιά διαστήματα, συστήματα συστάσεων, παιχνίδια hypercasual, εκμάθηση λεξιλογίου

Acknowledgments

I would like to warmly thank everyone who contributed to the completion of this thesis.

I thank my supervisor, Professor Nikolaos Korfiatis, for the guidance provided throughout this work, the members of the examination committee for their time, and all the professors of the Department of Informatics who contributed to my education during my undergraduate studies.

Special thanks go to my family and friends for their unwavering support, understanding and encouragement that they provided me throughout all these years of studies.

Corfu, September 2026

Mohammad Matin Marzie

Contents

A	Introduction	1
A.1	Motivation	1
A.2	Background	2
A.2.1	Short-form Video	2
A.2.2	Hypercasual and Hybrid-Casual Games	3
A.3	Problem Statement and Research Questions	4
A.4	Objectives, Scope, and Approach	5
A.4.1	Objectives	5
A.4.2	Scope and Delimitations	6
A.4.3	Methodological Approach	6
A.5	Significance	7
A.6	Thesis Structure	7
B	Literature Review	9
B.1	The Landscape of Mobile-Assisted Language Learning (MALL)	9
B.1.1	Market Evolution and Platform Proliferation	9
B.1.2	Critical Review and Systematic Limitations of Current Platforms	10
B.2	Pedagogical Foundations	12
B.2.1	Sociocultural Theory and the Zone of Proximal Development (ZPD)	13
B.2.2	The Input Hypothesis and the Lexical Coverage Threshold	13
B.2.3	Cognitive Load Theory and Microlearning	14
B.2.4	Memory Decay and Spaced Repetition Models	14
B.2.4.1	<i>The Ebbinghaus Forgetting Curve</i>	14
B.2.4.2	<i>Classic Leitner's Algorithm</i>	15

B.2.4.3	<i>Parametric Half-Life Regression</i>	15
B.2.4.4	<i>FSRS and the DHP Model</i>	16
B.2.5	Active Recall and the Testing Effect	19
B.3	Modalities for Language Acquisition	19
B.3.1	Short-Form Video in Second Language Acquisition (SLA)	19
B.3.1.1	<i>Incidental Vocabulary Acquisition through Audiovisual Input</i>	19
B.3.1.2	<i>The Gap: Passive Consumption without Reinforcement</i>	20
B.3.2	Serious Games and Gamification in Language Learning	21
B.3.2.1	<i>Game-Based Learning versus Gamification</i>	21
B.3.2.2	<i>Difficulty Adaptation in Hypercasual Games</i>	21
B.3.2.3	<i>Active-Recall Game Mechanics for Vocabulary</i>	21
B.4	Algorithmic Frameworks for Personalized Recommendation	22
B.4.1	Content-Based Filtering	22
B.4.2	Collaborative Filtering and Hybrid Recommender Systems	22
B.4.3	Recommender Systems in Educational Technology	23
B.5	Synthesis: A Theory-Driven Design Framework	23
C	System Architecture & Methodology	25
C.1	Design Requirements	25
C.1.1	Functional Requirements	25
C.1.2	Non-Functional Requirements	26
C.2	System Overview	27
C.3	Technology Stack	29
C.3.1	Frontend — React Native / Expo	29
C.3.2	Backend API — Node.js / Express	29
C.3.3	Reels Service — Python / FastAPI	30
C.3.4	Database — PostgreSQL	30
C.4	Data Modelling	30
C.4.1	Linguistic Content	30
C.4.2	User State	32
C.4.3	Engagement Signals	32
C.5	Onboarding and Learner-Model Initialisation	33
C.6	Recommendation Engine Design	33

<i>Contents</i>	ix
-----------------	----

C.6.1	Stage 1: Comprehensibility Filtering	34
C.6.2	Stage 2: Spaced-Repetition Prioritisation	34
C.6.3	Stage 3: Content-Based Personalisation	35
C.6.4	Combining the Three Stages: A Cascade, Not a Weighted Sum	35
C.7	Gamification Design	35
C.7.1	Progression Currencies: Coins, Energy, and Experience	36
C.7.2	Design Principle: Every Game Round Doubles as an FSRS Review	36
C.8	Vocabulary-Driven Games	37
C.8.1	Wordle-Style Game	37
C.8.2	Word of Wonders (Crossword)	38
C.9	Design Decisions and Trade-offs	38
C.9.1	Offline-First Client State	38
C.9.2	Memory State on the Device	39

D Implementation Details 40

D.1	Database Schema	40
D.1.1	Linguistic Content Tables	40
D.1.2	User State Tables	41
D.1.3	Engagement Signal Tables	41
D.2	Backend API	41
D.2.1	Authentication and Authorization	42
D.2.1.1	JWT Access/Refresh Token Rotation	42
D.2.1.2	Google OAuth 2.0	42
D.2.1.3	Password Reset and Request Limiting	43
D.2.2	Registration and Vocabulary Initialisation	43
D.2.2.1	Proficiency-Based Vocabulary Auto-Seeding	43
D.2.3	Offline Synchronisation	44
D.2.3.1	Change-Set Ordering: Deletes, Updates, Inserts	44
D.2.4	Dictionary and Letters Modules	44
D.2.5	Account Deletion and Personal Data	45
D.3	Frontend Architecture	45
D.3.1	Navigation and Screen Structure	45
D.3.2	State Management: Context and Hooks	46

D.3.3	Offline-First Strategy	46
D.3.4	Practice Reminders	47
D.3.5	Grammar Video Library	48
D.4	Reels Service	48
D.4.1	Service Architecture	48
D.4.2	Reel Creation and Dialogue Enrichment	49
D.4.3	Recommendation Engine Implementation	49
D.4.3.1	Stage 1: Candidate Loading and Comprehensibility Scoring	49
D.4.3.2	Stage 2: Frontend-Supplied Due-Word Partitioning	50
D.4.3.3	Stage 3: TF-IDF Profile Vectors and Cosine Similarity	50
D.4.4	Personalisation Implementation Status	51
D.5	Game Modules	51
D.5.1	Wordle-Style Game Implementation	51
D.5.2	Word of Wonders: Procedural Crossword Generation	52
D.5.2.1	Word Placement and Intersection Search	52
D.5.2.2	Letter-Budget and Word-Priority Heuristics	52
D.5.2.3	Due-Word Injection as the Practice Word	52
D.6	Deployment and Distribution	54
E	Evaluation & Results	55
E.1	Evaluation Approach	55
E.1.1	Research Questions and Evidence	55
E.1.2	Data and Reproducibility	56
E.2	Component-Level Algorithmic Evaluation	57
E.2.1	Word of Wonders: Grid-Generation Benchmark	57
E.2.1.1	Method	57
E.2.1.2	Robustness	57
E.2.1.3	Words Placed	58
E.2.1.4	Qualitative Example: On-Device Boards Across CEFR Levels	59
E.2.1.5	Generation Time	59
E.2.2	Recommendation Engine: The Cold-Start Problem	61
E.2.3	Recommendation Engine: Stage 1 Comprehensibility Filter	62
E.2.4	Recommendation Engine: Stage 2 Due-Word Prioritisation	63

<i>Contents</i>	xi
E.2.5 Recommendation Engine: Stage 3 Content-Based Ranking	65
E.3 Backend Correctness Testing	65
E.4 Discussion and Threats to Validity	67
E.4.1 What the Results Show	67
E.4.2 Threats to Validity	67
F Conclusions & Future work	69
F.1 Contributions	69
F.1.1 Answers to the Research Questions	70
F.2 Limitations	71
F.2.1 Evidence and Content	71
F.2.2 Language Processing	72
F.2.3 Recommendation and Progression	72
F.2.4 Onboarding and Social Scaffolding	73
F.3 Future Work	74
F.4 Concluding Remarks	76
Bibliography	78
Abbreviations	83
Glossary	85

List of Figures

B.1	The forgetting curve, with original data from Ebbinghaus [1], as replotted by Murre and Dros [2]. Image: Jaap M. J. Murre, Wikimedia Commons, CC BY 4.0. . . .	15
B.2	The classic five-box Leitner scheduling system. Image: Zirguezi, Wikimedia Commons [3], CC0 1.0.	16
C.3	High-level system architecture: the mobile frontend communicates with the Backend API (<code>api.glosy.gr</code>) and the Reels Service (<code>recs.glosy.gr</code>) over authenticated REST endpoints; both services share a single PostgreSQL instance. .	28
C.4	Entity-relationship diagram of the 21-table schema, grouped into the three axes of Section C.4: Linguistic Content, User State, and Engagement Signals.	31
E.5	Mean words placed on the board ($\text{cap} = 12$) as a function of simulated tracked-vocabulary size, per language. Error bars show ± 1 standard deviation over 200 repetitions per point.	58
E.6	Word of Wonders boards generated on a physical device for an English learner who declared each of the six CEFR levels at onboarding, all other conditions held fixed. Board density visibly grows from a bare two-word cross at A1 to a multi-armed board of several intersecting words by B2–C2.	60
E.7	Mean grid-generation wall-clock time (ms) as a function of simulated tracked-vocabulary size, per language, measured under Node.js/V8 on a development machine. Error bars show ± 1 standard deviation over 200 repetitions per point. . . .	61
E.8	The Personalization screen (<code>PersonalizationSlide.tsx</code>), the last onboarding page	63

E.9 Share of the 50 tokenised Farsi reels passing Stage 1, per onboarding level, at the threshold the application uses (0.98) and, for comparison only, a looser one (0.5). Each level’s known vocabulary is what registration seeds for it (Section D.2.2); level N receives only four starter words. 64

List of Tables

<i>B.1 Comparative feature matrix: Established LLPs vs. Glosy framework.</i>	<i>12</i>
<i>B.2 Theory-driven design traceability matrix: theoretical foundation, design implication, and the corresponding Glosy component.</i>	<i>24</i>
<i>C.1 Functional requirements, the source of each, and where it is realised.</i>	<i>26</i>
<i>C.2 Non-functional requirements, the source of each, and where it is realised.</i>	<i>27</i>
<i>E.1 Evidence gathered for each research question, and what remains untested.</i>	<i>56</i>

Chapter A

Introduction

A.1 Motivation

The scale of foreign language acquisition is an unprecedented phenomenon in contemporary applied linguistics. In our globalised society, over 1.5 billion individuals are actively studying a foreign language [4], seeking to facilitate cross-cultural communication, academic mobility, and access to global scientific literature. This widespread interest in language learning in conjunction with the advent of technological advancement (e.g., smartphones, AI) has shifted the paradigm of second language acquisition (SLA) away from traditional classroom-bound instruction toward self-directed, digital learning environments. Currently, independent, mobile-assisted platforms represent the most widespread medium for independent language study, enabling millions of learners to engage with language materials on demand [5].

Despite the ubiquity of mobile-first access and digital resources, contemporary Mobile-Assisted Language Learning (MALL) faces critical pedagogical efficacy bottlenecks. While self-learning mobile applications have become the standard interface for independent L2 (second language) acquisition [5], their long-term educational utility is severely compromised by learner attrition. Industrial Analysis of self-directed language learning applications reveals a severe pedagogical retention deficit, with first-week user attrition rates exceeding 60% across many major platforms [5].

Furthermore, modern lifestyle demands and cognitive workloads compress the time available for structured, long-form study [6]. Traditional, exhaustive learning structures fail to integrate into the daily routines of adult learners, as evidenced by empirical findings indicating that 53% of active

adult learners and half of educational directors identify excessive daily workloads as the primary barrier to ongoing instruction [6]. To circumvent these temporal constraints, learner behaviors have increasingly shifted toward autonomous, bit-sized micro-learning formats.

While modern mobile applications have successfully popularised the micro-learning format, current digital implementations remain heavily constrained by structural and pedagogical bottlenecks. As analyzed in depth within the literature review (Chapter B), existing MALL platforms frequently rely on uniform, static curricula that fail to adapt to individual learner demographics, lack authentic multimedia integration, and reduce gamification to repetitive, decontextualized translation drills—a phenomenon termed “shallow gamification.” Consequently, these platforms fail to support advanced proficiency development or sustain long-term cognitive engagement. This systemic deficit shifts the scientific inquiry: *how can mobile language learning frameworks transcend these repetitive, drill-based limitations to deliver adaptive, culturally authentic, and cognitively immersive media that aligns with the organic attention habits of modern learners?* Addressing this question requires investigating informal digital environments where contemporary learners willingly and habitually direct their attention.

A.2 Background

To address these completion and engagement deficits, modern Mobile-Assisted Language Learning (MALL) must align with the media consumption realities of contemporary learners. This thesis explores the integration of two prominent digital phenomena: short-form video and hypercasual game mechanics.

A.2.1 Short-form Video

Short-form video is a media format native to contemporary social media platforms, characterized by its brevity—typically ranging from 15 seconds to 3 minutes—and optimized for rapid, algorithmically driven consumption.

Short-form video has become the dominant interface of digital attention across social networks. Statistically, YouTube Shorts has achieved over 200 billion daily views [7], while Meta Platforms announced: “Video is a particular bright spot, with video time spent on Instagram up more than 30% since last year and 5% more time spent on Facebook” [8]. TikTok’s global active advertising audience has reached 1.59 billion adults, representing 27.5% of the global population aged 18 and

older [9].

This media format captures a disproportionate share of mobile attention. Globally, TikTok Android users average 1 hour and 37 minutes per day in the application, compared with 1 hour and 25 minutes for YouTube, 1 hour and 13 minutes for Instagram, and 1 hour and 7 minutes for Facebook [10].

This behavioral pattern is heavily concentrated among younger cohorts: 75% of US teens visit YouTube daily, and 61% engage with TikTok daily [11]. Among US adults, daily TikTok usage exhibits a sharp ten-to-one generational gap, captured by 50% daily usage among 18-to-29-year-olds compared to just 5% among those aged 65 and older [12].

Pedagogically, the cognitive appeal of short-form video lies in its brevity and high visual density. While platforms keep raising video length caps (with TikTok allowing up to 10 minutes and Reels up to 3 minutes) [13], consumer behavior remains heavily anchored around rapid completion. Wistia’s benchmark analysis of 13 million videos reveals that videos under 60 seconds hold a 52% average engagement rate, meaning viewers watch more than half the runtime before scrolling [14]. In contrast, longer videos see completion rates decay exponentially [14, 15]. Short-form videos therefore represent a highly engaging, authentic delivery mechanism for Second Language Acquisition (SLA).

A.2.2 Hypercasual and Hybrid-Casual Games

Hypercasual video games are defined as highly accessible, mobile-native software products characterized by minimalist user interfaces, instantaneous “tap-to-play” mechanics, and extremely short core gameplay loops [16]. Unlike casual games, which require some level of long-term planning, tutorial acquisition, or system familiarity, hyper-casual games demand no pre-existing cognitive schema or learning curve from the user, relying on immediately intuitive controls (core game mechanics) such as direction and drawing, matching, puzzle, rising and falling, swerving, swiping, tapping/timing and word and board [16, 17]. In recent years, this genre has evolved into “hybrid-casual” gaming, which retains the low-barrier entry and core mechanics of hypercasual games while strategically incorporating mid-core elements, such as progressive meta-features, item collection, and light narrative layer systems [17].

The exceptional appeal and potential addictiveness of these games are central to their design and rapid market expansion [17]. Reflecting this widespread appeal, the hyper-casual and hybrid-casual genres have experienced exponential audience expansion globally, emerging as a dominant

leisure activity on both major mobile platforms [18]. The addictiveness of these games are historically highlighted. Flappy Bird, which was voluntarily withdrawn from download by its creator due to concerns over its addictive and habit-forming impact on players [17].

Furthermore, because they are playable primarily on ubiquitous mobile devices and utilize intuitive user interfaces, hypercasual games reach a broad demographic spectrum. Notably, they appeal strongly to populations that historically do not identify as gamers or have been marginalized by the “male gamer stereotype” [19]. These include female players—who comprise 63% of the mobile gaming market [20]—as well as older adults (aged 65+) seeking cognitive maintenance, mental health preservation, and familial bonding [21].

Despite this high volume of daily engagement, players primarily consume these games as simple digital leisure pursuits to pass time, alleviate boredom, or fill transitional moments during daily routines and commutes [16, 17]. The highly engaging, low-friction mechanics of hypercasual and hybrid-casual games represent a powerful, untapped attention-delivery system. While their mass appeal stems precisely from this cognitively undemanding activity, the same properties suggest a different use. Hypercasual games can already adapt their difficulty to the individual player’s observed performance, which improves retention and replay value [22] (Section B.3.2.2); the same adaptivity could be directed at what the player knows rather than only at how well they play. This raises the question: can the attention that players habitually spend passing time in hypercasual games be redirected toward language learning?

A.3 Problem Statement and Research Questions

Short-form video and hypercasual games already capture a large and habitual share of learners’ attention, yet neither is designed to teach a language. This thesis asks whether that attention can be redirected toward language learning, and how to make the result effective rather than merely engaging. Doing so raises three design problems. First, each learner’s vocabulary must be tracked word by word, so that the system knows which words the learner has met and which are due for review. Second, a recommendation algorithm must use this word-level state to select reels that are both comprehensible and lexically challenging. Third, each game level must be generated from the same word-level state, so that a round of play is also a round of practice.

The central research problem is the absence of a language learning system that simultaneously leverages (i) the authentic contextual richness of user-generated short-form video and (ii) gamified

reinforcement adapted to the individual learner’s vocabulary state.

Research questions:

- RQ1.** How can a mobile application integrate authentic short-form video content with a word-level vocabulary tracking model to support second language acquisition?
- RQ2.** How should a content recommendation engine be designed to select reels that are both comprehensible and lexically challenging for a given learner, based on forgetting curves?
- RQ3.** To what extent can hypercasual game mechanics, when parameterised by the learner’s vocabulary state, serve as effective spaced-repetition exercises within a language learning application?

A natural objection is that a learner could already acquire vocabulary by watching reels on existing social media platforms. Section B.3.1 shows why this is not sufficient: passive viewing delivers comprehensible input, but provides no tracked vocabulary state, no scheduled review, and no active recall.

This thesis proposes **Glosy**, a cross-platform mobile application that addresses the identified gaps by embedding vocabulary acquisition within short-form authentic video content and reinforcing it through personalised hypercasual games. The system tracks each learner’s vocabulary state at the word level, uses that state to select contextually appropriate reels, and builds the words of every game round from the same vocabulary model.

A.4 Objectives, Scope, and Approach

A.4.1 Objectives

The research questions above translate into four concrete objectives:

- O1.** Model each learner’s vocabulary word by word, holding a spaced-repetition card for every tracked word and linking it to the dictionary and to the subtitles of every reel (RQ1).
- O2.** Build a three-stage recommendation engine that filters reels for comprehensibility, prioritises those containing words due for review, and ranks the remainder by similarity to the learner’s viewing history (RQ2).

- O3.** Generate two vocabulary-driven games, a Wordle-style guessing game and a procedurally generated crossword, from the same vocabulary state, so that every round of play is also a spaced-repetition review (RQ3).
- O4.** Deliver these components as a working, offline-capable prototype for Android and iOS, and evaluate their algorithmic behaviour on real seeded content.

A.4.2 Scope and Delimitations

The work is deliberately bounded. The system’s own learning mechanics address vocabulary acquisition only. Grammar is not taught or tested within *Glosy*; it is supported only by a curated collection of external YouTube lessons, kept per learning language, which the learner can watch from the application’s Videos screen or open in YouTube. The application offers no pronunciation or speaking exercises. It supports three languages — English, Greek, and Farsi, the last a right-to-left script — and targets both Android and iOS from one codebase. The two platforms are at different stages of distribution: the Android build is published as versioned APK releases on GitHub, whereas the iOS build is currently available only through Expo. The thesis also treats content supply as outside its scope: the reel catalogue is small and was seeded by a maintainer to demonstrate the recommender, not curated as a learning library (Section F.2). Most importantly, the thesis evaluates the system’s algorithmic components, not its effect on learners. No controlled study with learners has been conducted, so no claim is made about vocabulary gain, retention, or engagement, and the content-based ranking stage of the recommender, although implemented, is not tested (Chapter E).

A.4.3 Methodological Approach

The thesis follows a design-and-build approach. The literature review (Chapter B) yields the design principles: among them the 98% lexical-coverage criterion for comprehensible input and a spaced-repetition model of memory decay. Chapters C and D turn these principles into an architecture and a working implementation, and Chapter E tests the result. Because no learner has yet used the system, the evaluation examines each component directly. The crossword generator is benchmarked over 4,200 trials on word pools sampled from the seeded dictionaries, and the first two recommender stages are run on the real reel catalogue. The evaluation scripts are part of the project repository, and the crossword benchmark can be repeated from it in full. The recommender experiments also

need the reel catalogue, which is not published (Section E.1.2).

A.5 Significance

The significance of this work lies in three areas.

For learners, *Glosy* turns time that is already spent on short videos and casual games into vocabulary practice, so that learning does not require finding separate study time — the workload barrier identified above. It is designed mobile-first, for the device on which those habits already take place, and it supports offline gameplay and practice, which serves learners who study where internet access is limited, a use case that market analyses identify for offline learning features [23].

For research, it operationalises comprehensible input as a measurable criterion: the recommender keeps only reels in which the learner already knows at least 98% of the words, which turns a largely qualitative construct into a machine-checkable one (Section B.2.2). It also links a single word-level memory model to both content selection and play, so that watching a reel and completing a game round each act as a spaced-repetition review.

For practice, it is a working prototype rather than a design proposal, and its architecture is language-agnostic: adding a language requires content ingestion, not code changes (Chapter F). It currently supports English, Greek, and Farsi, including a right-to-left script.

Because no controlled study with learners has yet been conducted (Chapter E), these are benefits the design is built to deliver rather than effects it has been shown to produce.

A.6 Thesis Structure

The remainder of this thesis is organised as follows. Chapter B reviews mobile-assisted language learning platforms and their limitations, and establishes the pedagogical, memory-modelling, and recommendation foundations from which the design follows. Chapter C presents the system architecture, technology stack, data model, and the design of the recommendation engine and the two games. Chapter D documents the implementation of each component — database schema, backend API, frontend, reels service, and game modules — with reference to the source code. Chapter E evaluates the system’s algorithmic components, namely the Word of Wonders level generator and the recommendation engine, since no controlled study with learners has yet been conducted. Chap-

ter F states the contributions, acknowledges limitations, and proposes future work.

Chapter B

Literature Review

B.1 The Landscape of Mobile-Assisted Language Learning (MALL)

B.1.1 Market Evolution and Platform Proliferation

Digital language learning platforms (LLPs) have become dominant in foreign language education, with over 70 major commercial applications serving millions of active learners through Mobile-Assisted Language Learning (MALL) [24]. The scale attained by the market leaders illustrates the extent of this concentration: Duolingo alone reports 58.7 million daily active users and 140.6 million monthly active users as of mid-2026 [25]. A single platform’s design choices therefore shape the learning experience of a substantial share of the world’s self-directed language learners, which makes a critical examination of the dominant design paradigm — rather than only of usage statistics — a necessary starting point for this thesis.

This rapid expansion has been further accelerated by Generative Artificial Intelligence (AI) and Large Language Models (LLMs), which have streamlined content authoring and curriculum scaling. While authoring the first 100 language courses on Duolingo required over 12 years of manual pedagogical design, generative models facilitated the deployment of 148 additional courses in early 2025 alone [26]. Content authoring is consequently no longer the primary bottleneck to platform growth; as Section B.1.2 argues, the bottleneck has shifted to *pedagogical design* — how the abundant, cheaply-generated content is sequenced, contextualised, and reinforced for the individual learner.

B.1.2 Critical Review and Systematic Limitations of Current Platforms

Karasimos [24] conducted a cross-platform review of five major commercial LLPs — Duolingo, Memrise, Busuu, LingQ, and Babbel — evaluated against a fixed set of pedagogical and usability criteria. Although several of these platforms have iterated substantially since that 2022 review, the underlying architectural pattern it identifies — a static exercise bank gamified with points, streaks, and leaderboards — remains the dominant paradigm in 2026. Building on that review and on our own examination of the current (2026) state of these five platforms, six systematic limitations recur across the category:

1. **Uniform Content Delivery Across Learner Demographics and Interests:** Existing platforms deliver static, uniform curricula that do not account for variations in age cohorts (e.g., Gen Z, adults, seniors), communicative objectives, or cultural nuances. Nor do they account for variation in individual *interest*: two learners with identical proficiency levels but distinct interests — one following football, the other following technology news — are shown the same fixed exercise bank rather than content drawn from the topics that would sustain their engagement. Machine learning implementations in mainstream platforms focus predominantly on tuning exercise *difficulty* rather than personalising semantic *content* to either dimension [23]; that is, adaptivity is applied to *how hard* an item is, not to *who it is for* or *what it is about*.
2. **Insufficient Multimedia Integration:** Primary instructional routines rely predominantly on isolated text prompts and synthetic text-to-speech audio drills. This structure underutilises the visual and paralinguistic memory channels — gesture, facial expression, intonation, situational context — that reinforce natural conversational patterns and prosody, and that a short video clip conveys for free alongside the spoken utterance.
3. **Absence of Authentic, Real-World Linguistic Context:** Authentic input — language produced by native speakers for native speakers within a genuine communicative context, rather than written or graded for pedagogical consumption — is widely regarded as essential for developing communicative competence, and its use is associated with increased learner motivation and cultural awareness [27]. Mainstream LLP curricula, by contrast, frequently isolate target vocabulary into synthetic, decontextualised sentences authored specifically for the exercise. Authentic materials are not a costless substitute, however: they routinely presuppose cultural background knowledge that a foreign learner does not yet possess, which can

B.1. The Landscape of Mobile-Assisted Language Learning (MALL) 11

impede comprehension [28]. Existing LLPs offer no mechanism through which a confused learner can resolve this kind of gap in situ (Section B.1.2, item 6, returns to this point).

4. **Shallow Gamification:** Nearly all mainstream platforms are *gamified*, in the sense of layering points, streaks, badges, and leaderboards — game-design elements — onto an otherwise conventional drill [29]. Few, however, qualify as *game-based learning*, in which the learning content is embedded within the core mechanical loop of an actual game rather than merely rewarded by one [30]. Duolingo’s exercises, for instance, remain multiple-choice or sentence-construction drills regardless of how many streak-freeze tokens or leaderboard leagues surround them: the reward layer is decorative rather than structural. This distinction matters pedagogically, because points and streaks primarily sustain *extrinsic* engagement (returning to the app), whereas a well-designed game mechanic can additionally scaffold the *cognitive act of retrieval itself*. Section B.3.2 develops this distinction as a pedagogical theory; Chapter C operationalises it in the design of Glosy’s vocabulary-driven games.
5. **Limited Advanced Proficiency Support (B2+):** Current mobile platforms focus heavily on A1–B1 grammar foundations, offering sparse resources for upper-intermediate and advanced learners requiring idiomatic fluency, register variation, and natural speech rates. Except for LingQ, which provides authentic content, most platforms fail to support learners meaningfully beyond the B1 level [24].
6. **Absence of Sociocultural Grounding:** Vygotsky’s sociocultural account of cognitive development holds that learning is fundamentally a socially mediated process, not a solitary one (Section B.2.1) [31, 32]. Current mainstream platforms, however, are architected predominantly around single-player, drill-and-feedback loops with limited collaborative scaffolding: a learner who does not understand an idiom or a cultural reference has no in-context avenue to ask a native speaker, a peer, or an instructor. Busuu is a partial exception among the five platforms reviewed here, offering a community feature in which native speakers correct submitted writing and speech samples; this is a genuine sociocultural mechanism, but it is decoupled from the platform’s own exercises rather than attached to the specific piece of content the learner is struggling with. Outside the five platforms formally reviewed in this section, dedicated language-exchange applications such as HelloTalk [33] address the social-scaffolding gap directly, connecting learners in real time with native speakers for authentic conversation and correction. This is precisely the gap that authentic materials expose in item 3 above — authentic content increases the *need* for social scaffolding at exactly the

point where a structured, vocabulary-tracking curriculum platform provides the *least* of it — and it is the gap *Glosy* targets by pairing a tracked vocabulary model with comment-driven, content-attached social scaffolding, identified as future work in Chapter F.

Table B.1 summarises the resulting comparative feature matrix, contrasting the five established platforms reviewed above with the design targeted by *Glosy*.

Table B.1: Comparative feature matrix: Established LLPs vs. *Glosy* framework.

Feature / Dimension	Duolingo	Memrise	Busuu	Babbel	LingQ	Glosy
Personalized / Interest-Driven Content	Low	Low	Medium	Medium	High	High
Authentic Multimedia Video	Low	Medium	Medium	Low	High	High
Deep Gameplay Mechanics	Low	Low	Low	Low	None	High
B2+ Proficiency Support	Low	Low	Medium	Low	High	High
Adaptive SRS Tracking	High	High	Medium	Medium	Medium	High
Sociocultural / Social Scaffolding	Low	Low	Medium	Low	Medium	High
Open-source	No	No	No	No	No	Yes
Novice (A0–A1) Onboarding Ease	High	High	High	High	Low	Low

The remainder of this chapter builds the theoretical foundation from which the *Glosy* design responds to each of these six limitations: Section B.2 establishes the pedagogical theory, Section B.3 reviews the two content modalities (short-form video and serious games) through which that theory is delivered, and Section B.4 reviews the algorithmic frameworks that select and sequence content for the individual learner. Section B.5 closes the chapter by mapping each theoretical foundation explicitly onto a corresponding design decision realised in Chapter C.

B.2 Pedagogical Foundations

This thesis follows a *theory-driven design* approach: every major design decision documented in Chapter C is traceable to one of the theoretical foundations established in this section, rather than to intuition alone. This section is deliberately scoped to *theory* — what the literature says about how second language vocabulary is acquired, retained, and forgotten — and defers the specific computational mechanisms by which *Glosy* operationalises each theory to Chapter C, which references back to the relevant subsection below at each design decision.

B.2.1 Sociocultural Theory and the Zone of Proximal Development (ZPD)

Vygotsky's Sociocultural Theory posits that cognitive development occurs through socially mediated communication and cultural artefacts, wherein language functions as the primary cognitive tool for organising thought [31]. Central to this paradigm is the **Zone of Proximal Development (ZPD)**: the gap between a learner's independent competence — what they can accomplish unaided — and their potential development when assisted by a more capable interlocutor or by structured scaffolding. Lantolf's application of sociocultural theory to second language acquisition (SLA) re-frames the ZPD specifically for language learning: instructional stimuli should be pitched neither so far below the learner's current competence that no new learning occurs, nor so far above it that the material is incomprehensible without assistance [32]. Instructional design is optimal, in this account, precisely at the boundary of what the learner can process with appropriate support.

The ZPD is a qualitative, dyadic construct: it presupposes a more knowledgeable other (a teacher, a peer, a native speaker) providing scaffolding in real time. Section B.2.2 introduces a complementary, quantitative account of the same boundary — Krashen's Input Hypothesis — which *Glosy* operationalises computationally in the absence of a live interlocutor (Chapter C, Section C.6). The sociocultural dimension of the ZPD — social scaffolding through peer or native-speaker interaction — is not currently implemented in *Glosy*, and is identified as future work in Chapter F in direct response to the gap identified in Section B.1.2, item 6.

B.2.2 The Input Hypothesis and the Lexical Coverage Threshold

Krashen's Input Hypothesis proposes that language acquisition occurs when a learner is exposed to *comprehensible input* slightly beyond their current competence, denoted $i + 1$, where i represents the learner's current linguistic level [34]. Input that consists entirely of already-known language (i) offers no acquisitional value, while input too far beyond the learner's level ($i + n$, $n \gg 1$) is unparseable and therefore equally unproductive. Although formulated independently of Vygotsky, the Input Hypothesis is widely read alongside sociocultural theory in the SLA literature as a compatible, learner-internal account of the same comprehensibility boundary that the ZPD describes at the level of social interaction [32].

Unlike the ZPD, the $i + 1$ boundary admits an empirical, corpus-based operationalisation at the lexical level. Hu and Nation's corpus-based study of unsimplified text established that a reader requires knowledge of approximately 98% of the running word tokens in a text to comprehend it

unaided, with comprehension degrading sharply below that threshold [35]. This figure was subsequently popularised and synthesised as a general vocabulary-learning target in Nation’s widely-cited *Learning Vocabulary in Another Language* [36], and it remains one of the few points at which the largely qualitative construct of comprehensible input admits a precise, machine-checkable definition. This is the specific theoretical result that Chapter C, Section C.6 (Stage 1) operationalises as the comprehensibility filter applied to candidate reels.

B.2.3 Cognitive Load Theory and Microlearning

Cognitive Load Theory (CLT) holds that working memory capacity is strictly bounded — classically estimated at 7 ± 2 discrete chunks of information [37] — and that instructional design must minimise *extraneous* cognitive load (load imposed by poor presentation, unrelated to the material itself) in order to leave sufficient working-memory capacity for *germane* load, the productive cognitive work of constructing new long-term memory schemata [38]. Applied to instructional sequencing, CLT implies that a single learning episode should be bounded in scope: introducing too many new lexical items, grammatical structures, or task demands within one episode exceeds working-memory capacity and displaces schema construction with the effort of merely holding the material in mind.

Microlearning — the pedagogical strategy of decomposing instruction into short, self-contained units rather than long-form sessions — is a direct instructional response to this bound: by construction, a short unit cannot introduce more new material than the working-memory ceiling permits. This principle underlies the length constraints Chapter C imposes on both the video content and the game sessions in *Glosy*: each unit of engagement is scoped narrowly enough that the germane load it imposes remains tractable, consistent with the CLT account above.

B.2.4 Memory Decay and Spaced Repetition Models

B.2.4.1 The Ebbinghaus Forgetting Curve

Ebbinghaus’s foundational experiments on nonsense-syllable recall demonstrated that memory retention decays over time in the absence of reinforcement, most rapidly immediately after initial exposure and more slowly thereafter [1]. Although Ebbinghaus’s own savings-method data did not yield a single closed-form expression, later reconstructions commonly approximate the resulting *forgetting curve* as a negative exponential function of elapsed time t and a memory-strength

parameter S : retention $R(t) = e^{-t/S}$, where a larger S corresponds to a more durably consolidated memory. This exponential-decay account is the shared theoretical ancestor of every spaced-repetition model reviewed below: each attempts, with increasing sophistication, to estimate the parameters of an individual learner’s own decay curve for an individual item, and to schedule review immediately before predicted recall failure.

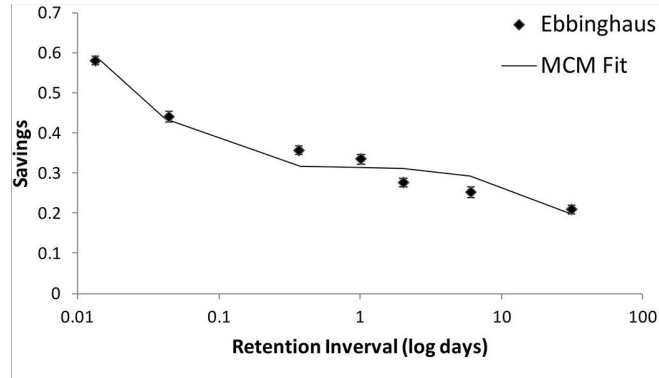


Figure B.1: The forgetting curve, with original data from Ebbinghaus [1], as replotted by Murre and Dros [2]. Image: Jaap M. J. Murre, Wikimedia Commons, CC BY 4.0.

B.2.4.2 Classic Leitner’s Algorithm

The Leitner system operationalised the forgetting curve as a simple, non-parametric scheduling heuristic decades before it became computationally tractable to fit an individual decay curve per item [39]. Flashcards are sorted into a series of boxes; a card answered correctly is promoted one box forward, to be reviewed at a longer interval, while a card answered incorrectly is reset all the way back to the first box, regardless of how far it had previously advanced, to be reviewed again at the shortest interval. The scheme requires no numerical memory model at all — the box number is a coarse, discretised proxy for the item’s memory strength S — which makes it trivially implementable but insensitive to the fact that different items, and different learners, decay at markedly different rates even within the same box.

B.2.4.3 Parametric Half-Life Regression

Duolingo’s half-life regression (HLR) model, introduced in 2016 [40], replaces the Leitner system’s discrete boxes with a continuous, per-item, per-learner estimate of the memory half-life h —

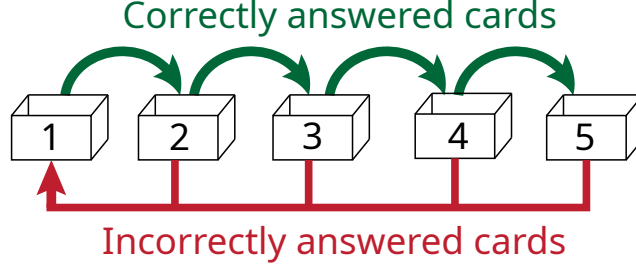


Figure B.2: The classic five-box Leitner scheduling system. Image: Zirguezzi, Wikimedia Commons [3], CC0 1.0.

the elapsed time at which recall probability has decayed to 50%. Recall probability p for a given item is modelled as an exponential decay in the elapsed interval Δt since it was last reviewed, directly generalising the Ebbinghaus curve of Section B.2.4.1:

$$p = 2^{-\Delta t/h} \quad (\text{B.1})$$

The half-life h itself is not fixed but estimated per item and per learner via a log-linear regression over a feature vector $\mathbf{x}$ — such as the number of prior correct and incorrect attempts — with a learned weight vector Θ :

$$\hat{h}_{\Theta}(\mathbf{x}) = 2^{\Theta \cdot \mathbf{x}} \quad (\text{B.2})$$

generalising the Leitner box index into a continuously-valued, learner- and item-specific quantity. The weight vector Θ is not known in advance: it must itself be fitted offline, by regression over a large historical corpus of review outcomes pooled across many users, before Equation B.2 can be applied to a single new one — Settles and Meeder trained their original model on Duolingo’s own multi-million-user interaction log [40]. A newly-launched application, without an equivalent corpus of prior review data from which to fit Θ , cannot reproduce this regression from its own history alone. Section B.2.4.4 reviews an alternative memory model that avoids this cold-start requirement.

B.2.4.4 FSRS and the DHP Model

The Free Spaced Repetition Scheduler (FSRS) originates in the DHP model developed at MaiMemo [41], a variant of what the open-source scheduling community terms the **DSR** model: memory state

is represented not by a single half-life h , as in HLR, but by three jointly-updated variables — **difficulty** $D \in [1, 10]$ (how intrinsically hard the item is to learn), **stability** S (the number of days for retrievability to decay to 90%), and **retrievability** R (the instantaneous, momentary probability of successful recall, analogous to p in HLR). Each is revised after every individual review rather than re-estimated from scratch, and each review is scored on a four-point grade $G \in \{1, 2, 3, 4\}$ (*again, hard, good, easy*). Since its first release, FSRS has been refined through several major versions (v1 through the current FSRS-6), each adding parameters and improving the empirical fit of its forgetting curve and update rules [42]; the formulation below is the current one (FSRS-6), noting where it revises earlier versions.

The forgetting curve. Rather than the fixed exponential decay of Equation B.1, FSRS models retrievability as a power-law function of the elapsed interval t and the current stability S , with a trainable decay exponent w_{20} (the w_i denote the elements, 0-indexed, of a fitted 21-parameter weight vector $\mathbf{w}$):

$$R(t, S) = \left(1 + \text{factor} \cdot \frac{t}{S}\right)^{-w_{20}}, \quad \text{factor} = 0.9^{-1/w_{20}} - 1, \quad (\text{B.3})$$

where the factor term is chosen precisely so that $R(S, S) = 0.9$ always holds, preserving stability’s definition regardless of the fitted value of w_{20} . This is itself the product of two earlier simplifications: the original FSRS v4 fixed the exponent at $w_{20} = 1$ with factor = 1/9, reducing Equation B.3 to $R(t, S) = (1 + t/9S)^{-1}$; FSRS-4.5 refined this to a fixed exponent of 0.5; only from FSRS-5 onward is the decay shape itself learned from data.

Initial state and difficulty updates. A word’s very first review initialises stability and difficulty directly from the grade it receives:

$$S_0(G) = w_{G-1}, \quad D_0(G) = w_4 - e^{w_5(G-1)} + 1, \quad (\text{B.4})$$

so that, for instance, an item first rated *again* ($G = 1$) starts at the lowest stability w_0 and the highest difficulty $D_0(1) = w_4$. On every subsequent review, difficulty drifts by a grade-dependent increment and is then pulled back towards a fixed target via *mean reversion*:

$$D' = D - w_6(G - 3) \cdot \frac{10 - D}{9}, \quad D'' = w_7 D_0(4) + (1 - w_7) D', \quad (\text{B.5})$$

anchoring difficulty to $D_0(4)$ (the initial difficulty of an *easy*-rated item) rather than letting it drift unboundedly after a long run of easy or hard grades — informally termed avoiding “ease hell” in the flashcard-scheduling community.

Stability updates. Following a successful review ($G \in \{2, 3, 4\}$), stability increases according to

$$S'_r(D, S, R, G) = S \cdot \left(e^{w_8(11-D)S^{-w_9}(e^{w_{10}(1-R)} - 1) \cdot w_{15}^{[G=2]} \cdot w_{16}^{[G=4]}} + 1 \right), \quad (\text{B.6})$$

while a lapse ($G = 1$) instead resets stability to a smaller post-lapse value,

$$S'_f(D, S, R) = w_{11} D^{-w_{12}} ((S + 1)^{w_{13}} - 1) e^{w_{14}(1-R)}. \quad (\text{B.7})$$

Writing $S_{\text{inc}} = S'_r/S$ for the multiplicative stability gain of a successful review, three qualitative properties of Equation B.6 recur throughout the FSRS documentation and are directly interpretable in terms of the theory already established in this chapter: S_{inc} decreases as difficulty D increases (harder items consolidate more slowly); S_{inc} decreases as stability S itself increases (an already well-consolidated memory is harder to strengthen further, consistent with the diminishing-returns shape of the Ebbinghaus curve in Section B.2.4.1); and S_{inc} increases as retrievability R decreases at the time of review, i.e. a successful recall of an *almost-forgotten* item strengthens memory more than reviewing an item that was never at risk — the computational signature of the spacing effect.

Same-day reviews (FSRS-6). The formulas above govern reviews spaced at least a day apart. FSRS-6's specific contribution is a separate, more conservative update for a card reviewed again on the same day it was last seen:

$$S'(S, G) = S \cdot e^{w_{17}(G-3+w_{18})S^{-w_{19}}}, \quad (\text{B.8})$$

which grows quickly while S is still small and flattens as S grows, converging once $S_{\text{inc}} = e^{w_{17}(G-3+w_{18})S^{-w_{19}}} = 1$; earlier versions used the same functional form without the $S^{-w_{19}}$ damping term, allowing same-day repetition to inflate stability unrealistically.

This finer-grained decomposition is, at the time of writing, the most empirically accurate publicly documented model of spaced-repetition scheduling. Critically for a new application, all of Equations B.3–B.8 update the per-item state *incrementally, after each individual review*, using a fixed parametric rule rather than a regression re-fitted in a single offline pass over a large historical corpus, as HLR requires (Section B.2.4.3). A new item begins from the initial-state Equation B.4 and refines its own difficulty and stability purely from the learner's own subsequent review history, without needing Duolingo-scale historical data first; *Glosy* adopts the published default parameter vector $\mathbf{w}$ rather than fitting its own, precisely because it does not yet have the review corpus such a fit would require. This is the specific reason Chapter C, Section C.6 (Stage 2) adopts the FSRS

formulation, rather than HLR, as the basis of the `SRS_Urgency` scoring term in the recommendation engine: a word’s recommendation urgency rises as its predicted retrievability R (Equation B.3) falls.

B.2.5 Active Recall and the Testing Effect

A separate, complementary strand of cognitive psychology addresses not *when* to review an item, but *how*. The testing effect — robustly demonstrated across decades of memory research — shows that the act of actively retrieving an item from memory (as in a test or a recall task) produces substantially better long-term retention than an equivalent amount of time spent passively re-studying the same item [43]. This finding motivates a specific design constraint distinct from, and additional to, the scheduling question addressed by Section B.2.4: *when* an item is due for review, the review task itself should require productive retrieval (e.g., recalling and typing a word) rather than passive recognition (e.g., selecting a familiar-looking option among distractors). Section B.3.2 reviews game mechanics through this lens.

B.3 Modalities for Language Acquisition

Sections B.2.2–B.2.5 establish *what* an effective input and reinforcement episode should look like theoretically: comprehensible ($i+1$), authentic, cognitively bounded, spaced, and actively recalled. This section reviews the empirical literature on the two content modalities through which *Glosy* delivers input and reinforcement respectively: short-form video, addressed to the input side of acquisition, and serious games, addressed to the active-recall side.

B.3.1 Short-Form Video in Second Language Acquisition (SLA)

B.3.1.1 Incidental Vocabulary Acquisition through Audiovisual Input

A substantial body of research in Computer-Assisted Language Learning (CALL) predates the short-form video format itself and concerns the closely related case of captioned television and film viewing. Vanderplank’s early work on teletext subtitles found that captioned video supports vocabulary and listening development beyond what audio alone provides, by allowing the learner to cross-reference the written and spoken forms of an unfamiliar item in real time [44]. More recent corpus-informed work confirms this incidental-learning effect at scale: Peters and Webb

found measurable incidental vocabulary gains from watching L2 television programmes without any explicit study task, with gains moderated by frequency of item occurrence and by the presence of captions [45], while Montero Perez, Peters, and Desmet found that enhancing target vocabulary within video captions (e.g., through highlighting) further improves retention relative to unenhanced captioned video [46]. Taken together, this literature establishes that passive audiovisual viewing is a genuine, if modest, channel of incidental vocabulary acquisition, and that the effect is strengthened, not weakened, by textual scaffolding synchronised to the audio track — precisely the token-level, timestamped transcript structure that a *Glosy* reel provides (Chapter C, Section C.4).

B.3.1.2 The Gap: Passive Consumption without Reinforcement

The incidental-acquisition literature reviewed above is consistent in one further respect: the vocabulary gains it reports from passive viewing alone are real but small, typically requiring many repeated incidental encounters of an item before it is durably retained [45]. Passive viewing, in other words, is an effective delivery mechanism for comprehensible input (Section B.2.2) but is not, by itself, a spaced-repetition system in the sense of Section B.2.4: a viewer has no mechanism by which an encountered item is tracked, scheduled for review, or actively retrieved once the video ends. This is the specific gap in generic short-form social video consumption — as opposed to a purpose-built application — that motivates the research question posed in Chapter A: watching unstructured reels on an existing social platform provides input without a tracked vocabulary state, and therefore without the scheduled, active-recall reinforcement that Sections B.2.4 and B.2.5 identify as necessary for durable retention. Closing this gap requires attaching a conventional SRS review step — of the kind ordinarily triggered by a static flashcard in an application such as Anki — directly to the moment a due item is encountered inside a reel, rather than leaving that encounter to pass by unrecorded. Chapter C details the interface through which *Glosy* does this: a due word appearing in a reel’s transcript prompts an in-context review step immediately after viewing, through which the learner’s recall grade G is captured and fed into the FSRS update rules of Section B.2.4.4 (Equations B.6–B.7) — the reel itself standing in for the static flashcard as the review’s triggering stimulus.

B.3.2 Serious Games and Gamification in Language Learning

B.3.2.1 Game-Based Learning versus Gamification

Section B.1.2 (item 4) distinguished *gamification* — decorating an existing task with game-design elements such as points and streaks [29] — from *game-based learning*, in which the instructional content is embedded within the mechanical loop of an actual game [30]. This distinction is well established as a specific research strand within CALL: Godwin-Jones’s review of games in language learning documents a persistent gap between the two, noting that commercially successful language-learning products have historically favoured drill-based gamification because it is comparatively cheap to build and easy to monetise, while games in which linguistic competence is itself the win condition remain comparatively rare [47]. Sykes and Reinhardt’s broader treatment of digital games in second and foreign language teaching similarly argues that a game’s pedagogical value derives from whether target-language production or comprehension is load-bearing to *winning the game*, not merely rewarded upon completion of an unrelated exercise [48].

B.3.2.2 Difficulty Adaptation in Hypercasual Games

Section A.2.2 (Chapter A) established the hypercasual genre’s core appeal: minimal onboarding, immediately intuitive mechanics, and short session length — properties that align closely with the microlearning argument of Section B.2.3. Within game design more specifically, dynamic difficulty adjustment (DDA) — algorithmically tuning a hypercasual game’s difficulty in response to the individual player’s observed performance, rather than fixing it in advance — has been shown to improve player retention and perceived replay value relative to a static difficulty curve [22]. This provides the game-design-theoretic counterpart to the lexical personalisation argument of Section B.4: just as a reel should be selected for lexical comprehensibility, a game level’s difficulty should be tuned to the player’s demonstrated skill — in *Glosy*’s case, the specific vocabulary items the level is built from, rather than a generic difficulty parameter.

B.3.2.3 Active-Recall Game Mechanics for Vocabulary

The testing-effect literature reviewed in Section B.2.5 [43] favours game mechanics that require *productive* retrieval of a target word’s orthographic form over mechanics that require only its *recognition* among distractors. Word-reconstruction puzzle formats — in which the player must recall

and produce a target word letter-by-letter, validating each attempt against positional or compositional constraints — structurally enforce this kind of active retrieval as the core win condition of the game itself, rather than as an optional or bypassable step. This is the specific theoretical property, rather than genre popularity alone, that motivates the word-reconstruction game modalities implemented in *Glosy* and documented in Chapter C, Section C.8.

B.4 Algorithmic Frameworks for Personalized Recommendation

Sections B.2 and B.3 establish *which* content is pedagogically appropriate for a learner at a given moment: comprehensible (Section B.2.2), due for review (Section B.2.4), and appropriately reinforcing (Section B.3). Selecting such content automatically, per learner, is a recommendation problem in the information-retrieval sense. This section reviews the two classical recommendation paradigms, content-based and collaborative filtering, and the strategies the literature offers for combining them; Section B.5 maps this theory onto *Glosy*'s own recommendation engine.

B.4.1 Content-Based Filtering

Content-based filtering recommends items by comparing an item's own features against a profile of a single user's own preferences, built from that user's history, without reference to other users' behaviour [49]. Its principal known limitation is that it has no way to express a temporal signal such as *when* a previously-seen item is due for review [49] — a gap the memory-decay models of Section B.2.4 are specifically designed to fill.

B.4.2 Collaborative Filtering and Hybrid Recommender Systems

Collaborative filtering (CF) recommends items based on patterns of agreement across many users' behaviour — favouring items that users with a similar engagement history have responded well to — rather than on an item's own features [49]. Because it depends on an interaction matrix populated across many users, CF is known to perform poorly under the *cold-start* problem, when a system has few users or little accumulated interaction history [49].

Burke's survey of hybridisation strategies for combining multiple recommendation techniques distinguishes several ways of doing so, including *weighted* hybridisation, in which each technique produces an independent score and the final score is their linear combination, and *cascade* hy-

bridisation, in which one technique coarsely ranks or partitions the candidate set and a second technique is applied only to break ties within it [50]. Ricci et al.’s handbook situates both strategies, and content-based and collaborative filtering more generally, within the broader design space of recommender systems [51].

B.4.3 Recommender Systems in Educational Technology

Recommender systems applied to e-learning typically personalise the sequencing of resources or exercise *difficulty* to a learner’s inferred ability, rather than the semantic content of what is recommended [52] — the same gap identified in Section B.1.2 (item 1).

B.5 Synthesis: A Theory-Driven Design Framework

This chapter has reviewed, in turn, the pedagogical theory of comprehension and retention (Section B.2), the empirical literature on the two content modalities through which *Glosy* delivers input and reinforcement (Section B.3), and the algorithmic frameworks by which content is selected and sequenced for the individual learner (Section B.4). Table B.2 makes explicit the traceability between each theoretical foundation reviewed above and the corresponding design decision it motivates in Chapter C, operationalising the theory-driven design approach stated at the outset of Section B.2.

Read together with the platform limitations identified in Section B.1.2, this synthesis reframes the research questions posed in Chapter A as a design problem with a specific theoretical answer for each of its parts: RQ1 is addressed by the content-based lexical filtering of Section B.4.1 operating over the short-form video modality of Section B.3.1; RQ2 is addressed by the cascade hybrid of Section B.4.2, fusing comprehensibility with the memory-decay models of Section B.2.4; and RQ3 is addressed by the active-recall game theory of Section B.3.2, parameterised by the same memory-decay state that governs RQ2. Chapter C now presents the system architecture that implements this framework.

Table B.2: Theory-driven design traceability matrix: theoretical foundation, design implication, and the corresponding Glosy component.

Theoretical Foundation	Design Implication	Glosy Component
ZPD (B.2.1) & Input Hypothesis (B.2.2)	Recommend content at the $i + 1$ comprehensibility boundary	Stage 1: Comprehensibility Filtering, 98% lexical coverage (Sec. C.6.1)
Cognitive Load Theory & Microlearning (B.2.3)	Bound each learning episode to a short, self-contained unit	90-second reel cap; Wordle attempt limit; Word of Wonders word/letter caps (Sec. C.2, Sec. C.8)
Forgetting Curve & Spaced Repetition (B.2.4)	Schedule review before predicted recall failure	Stage 2: Spaced-Repetition Prioritisation, due-word partitioning (Sec. C.6.2)
Testing Effect (B.2.5) & Game-Based Learning (B.3.2)	Require productive retrieval, not passive recognition	Wordle-style and Word of Wonders vocabulary games (Sec. C.8)
Cascade Hybrid Recommender (B.4.2)	Filter by comprehensibility, partition by due-for-review, break ties by engagement-weighted content similarity	Stage 1–3 cascade ranking (Sec. C.6.4)
Sociocultural Theory (B.2.1)	Provide social scaffolding for culturally-opaque authentic input	Collaborative learning features (Ch. F, Future Work)

Chapter C

System Architecture & Methodology

C.1 Design Requirements

The design of *Glosy* follows from three sources. The research questions of Chapter A state what the system must accomplish; the theoretical foundations reviewed in Chapter B and traced in Table B.2 state how it should do so; and the practical setting — a mobile application built and hosted on modest resources — sets limits on both. This section turns these sources into an explicit list of functional and non-functional requirements, so that each design decision in the rest of the chapter can be traced to the need it answers.

C.1.1 Functional Requirements

Table C.1 lists what the system must do. FR1 to FR4 correspond directly to the research questions, and FR5 to FR7 are the requirements a learner meets before any of them: choosing a language, arriving with an empty vocabulary, and entering the application without an account.

Table C.1: Functional requirements, the source of each, and where it is realised.

ID	Requirement	Source	Realised in
FR1	Track each learner’s vocabulary word by word, with a memory state that says when each word is next due	RQ1; spaced repetition (Sec. B.2.4)	User State (Sec. C.4.2)
FR2	Present authentic short videos whose transcripts are linked word by word to the dictionary	RQ1; short-form video (Sec. B.3.1)	Linguistic Content (Sec. C.4.1)
FR3	Recommend only reels the learner can largely understand, preferring those that contain words due for review	RQ2; comprehensible input (Sec. B.2.2)	Stages 1 and 2 (Sec. C.6.1, C.6.2)
FR4	Make every game round a review of words from the learner’s own vocabulary	RQ3; testing effect (Sec. B.2.5)	Game design (Sec. C.7.2)
FR5	Support English, Greek, and Farsi, including a right-to-left script	Multilingual scope of the thesis	Wordle-style game (Sec. C.8.1)
FR6	Initialise a new learner’s vocabulary without a placement test	Cold start (Sec. B.4.2)	Onboarding (Sec. C.5)
FR7	Let a learner start without an account and later carry that progress into one	First-week attrition (Chapter A)	Onboarding (Sec. C.5)

C.1.2 Non-Functional Requirements

Table C.2 lists the qualities the system must have. Several of them pull against the functional requirements, and the design settles each conflict explicitly rather than by accident.

Table C.2: Non-functional requirements, the source of each, and where it is realised.

ID	Requirement	Source	Realised in
NFR1	Keep every learning episode short and bounded	Microlearning (Sec. B.2.3)	90-second reel cap; game limits (Sec. C.2, C.8)
NFR2	Allow practice and play without connectivity, synchronising later	Learners with limited access (Chapter A)	Offline-first client (Sec. C.3.1, D.3.3)
NFR3	Degrade gracefully when data is sparse: no due words, no history, or few reels	Cold start (Sec. B.4.2)	Cascade ranking (Sec. C.6.4)
NFR4	Run on modest server hardware	512 MB reels-service host	CPU-only vectorisation (Sec. C.3.3)
NFR5	Serve Android and iOS from one codebase	Development resources	React Native and Expo (Sec. C.3.1)
NFR6	Add a new language through content, not code changes	Extensibility (Chapter F)	Language-independent schema (Sec. C.4)
NFR7	Use one login session across both back-end services	Usability and security	Shared token scheme (Sec. C.3.2, C.3.3)

Three of these conflicts shape the architecture. NFR4 rules out a heavy natural-language pipeline in the request path, so Stage 3 uses lightweight TF-IDF vectors and all linguistic preprocessing is done offline. NFR2 rules out scoring against server-side memory state, so the memory model runs on the device and the client tells the server which words are due. NFR1 caps the reel and game lengths even though longer units could carry more vocabulary. In each case the requirement that protects the learner’s time or access wins over the one that would add sophistication.

C.2 System Overview

Glosy is designed as a three-service architecture in which a mobile frontend, a Node.js backend API, and a dedicated Python reels microservice each own a distinct part of the system’s responsibilities, all three sharing a single PostgreSQL database. In development, the three services and the database are orchestrated together with Docker Compose: the backend API listens on port 3500, the reels service on port 3600, and PostgreSQL 18 on host port 5433 (mapped to its default port 5432 inside the container), each built and reloaded independently of the others so a change to one service does not require restarting the rest of the stack.

- **Frontend:** A cross-platform mobile application built with React Native and Expo Router, targeting Android and iOS from a single JavaScript/TypeScript codebase, and responsible for the offline-first client state, the reels feed, and the two vocabulary-driven games.
- **Backend API:** A Node.js/Express service responsible for authentication, user and vocabulary state, offline synchronisation, and the dictionary and letters modules.
- **Reels Service:** A Python/FastAPI microservice responsible for serving reels together with their structured dialogue data, and for the three-stage recommendation engine that selects and ranks them per learner.
- **Database:** A single PostgreSQL 18 instance, shared by both back-end services, whose schema encodes linguistic content, user state, and engagement signals (Section C.4).

Splitting the reels service from the backend API is a deliberate choice rather than an accident of the codebase’s history: the recommendation engine’s queries — joining a reel with its dialogue, sentences, per-token word data, and translations, then scoring the result against a learner’s vocabulary and engagement history — are naturally expressed as an object-relational query built up in code, which SQLAlchemy’s async ORM supports directly, whereas the backend API’s own queries are simple enough that raw parameterised SQL via the `pg` client remains the more direct choice. The two services therefore use different data-access strategies deliberately matched to the shape of the queries each one runs, rather than a single ORM standardised across the whole system.

A reel’s duration is capped at 90 seconds at upload time, rejected outright by the reel-creation schema if exceeded; this is a direct application of the microlearning bound of Section B.2.3, keeping a single unit of video input short enough that it cannot itself exceed a learner’s working-memory budget regardless of how much unfamiliar vocabulary it contains. Section C.8 describes the analogous

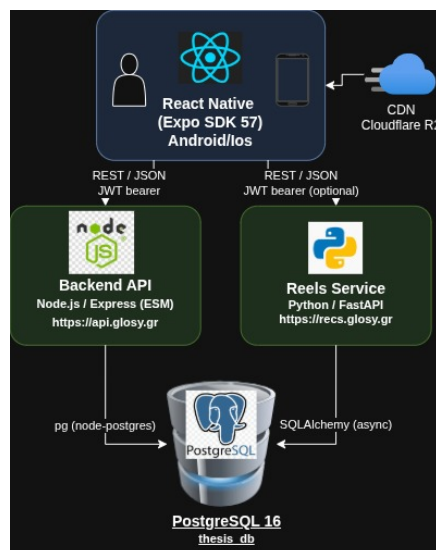


Figure C.3: High-level system architecture: the mobile frontend communicates with the Backend API (`api.glosy.gr`) and the Reels Service (`recs.glosy.gr`) over authenticated REST endpoints; both services share a single PostgreSQL instance.

bound on the other side of a learning episode, the games' own attempt and word-count limits.

C.3 Technology Stack

C.3.1 Frontend — React Native / Expo

The frontend is built with React Native 0.86 and React 19, on Expo SDK 57, using Expo Router for file-system-based navigation between the application's five tabs and its onboarding, settings, and game routes. `expo-video` handles reel playback, `expo-secure-store` holds authentication tokens, and React Native Reanimated (4.5) drives the gesture-based interactions in the reels feed and the two games, together with `@gorhom/bottom-sheet` for the swipe-to-review prompt described in Section C.7.2. The FSRS scheduling itself (Section C.6.4) runs entirely on-device via `ts-fsrs`, a published TypeScript implementation of the algorithm reviewed in Section B.2.4.4, rather than a bespoke implementation — there is no pedagogical reason to re-derive difficulty/stability/retrievability update rules whose reference implementation is both open-source and already exercised against the FSRS benchmark.

C.3.2 Backend API — Node.js / Express

The backend is a Node.js service on Express 4, communicating with PostgreSQL through the plain `pg` client rather than an ORM, since its own queries (per-row inserts/updates keyed by user and word id) do not benefit from an object-relational mapping layer the way the reels service's join-heavy queries do (Section C.2). Authentication combines JSON Web Tokens (short-lived access tokens, longer-lived refresh tokens, rotated via `jsonwebtoken`) with Google OAuth 2.0 (`google-auth-library`) as an alternative sign-in path; every request body is validated against a Joi schema before it reaches a controller, and the API is self-documented with Swagger/OpenAPI at `/swagger`. Video thumbnails are generated server-side with `fluent-ffmpeg` over a statically-bundled `ffmpeg` binary, and both the backend and the reels service read and write the same Cloudflare R2 (S3-compatible) object storage bucket through the AWS SDK's `S3Client`, so a reel's media is addressable identically regardless of which service produced it.

C.3.3 Reels Service — Python / FastAPI

The reels service is a FastAPI application over Pydantic 2 for request/response validation and SQLAlchemy 2's async ORM (via `asyncpg`) for the join-heavy queries described in Section C.2. It does not run its own authentication: instead, `python-jose` decodes the same HS256-signed access tokens the Node backend issues, against the same shared secret, so a single login session is valid across both services without a second sign-in step or a token-exchange call between them. Stage 3 of the recommendation engine (Section C.6.3) is implemented with scikit-learn's `TfidfTransformer` and `cosine_similarity`, a deliberately lightweight, CPU-only choice: the service runs on a 512 MB host, which rules out loading a full NLP pipeline (spaCy, Stanza, or Hazm) into every request worker purely to vectorise a reel's word-id bag. Sentence tokenisation and lemmatisation, which do need those heavier libraries, are instead performed offline by a separate backfill script rather than at request time, keeping the request path itself dependent only on scikit-learn's lightweight vectorisers.

C.3.4 Database — PostgreSQL

Both backend services share a single PostgreSQL 18 instance (Section C.2), chosen for its support of array operators, window functions, and common table expressions — used throughout the vocabulary bulk-seeding logic of Section D.2.2 — over a lighter-weight embedded database that would not support these directly in SQL. The full schema is detailed in Section C.4 and Section D.1.

C.4 Data Modelling

The schema is organised along the same three axes used throughout Chapter B's theoretical framework: content that exists independently of any learner, the state the system tracks about a specific learner, and the signals a learner generates while engaging with that content.

C.4.1 Linguistic Content

This axis holds everything a learner can be shown, independent of who they are: `languages`, `words` (each tagged with a `level` drawn from the CEFR scale A1–C2 plus a small number of non-CEFR codes used elsewhere in the schema) and their `word_translations`, `letters` for the alphabet-drill dictionary entries, `sentences` with per-token (`sentence_tokens`) and

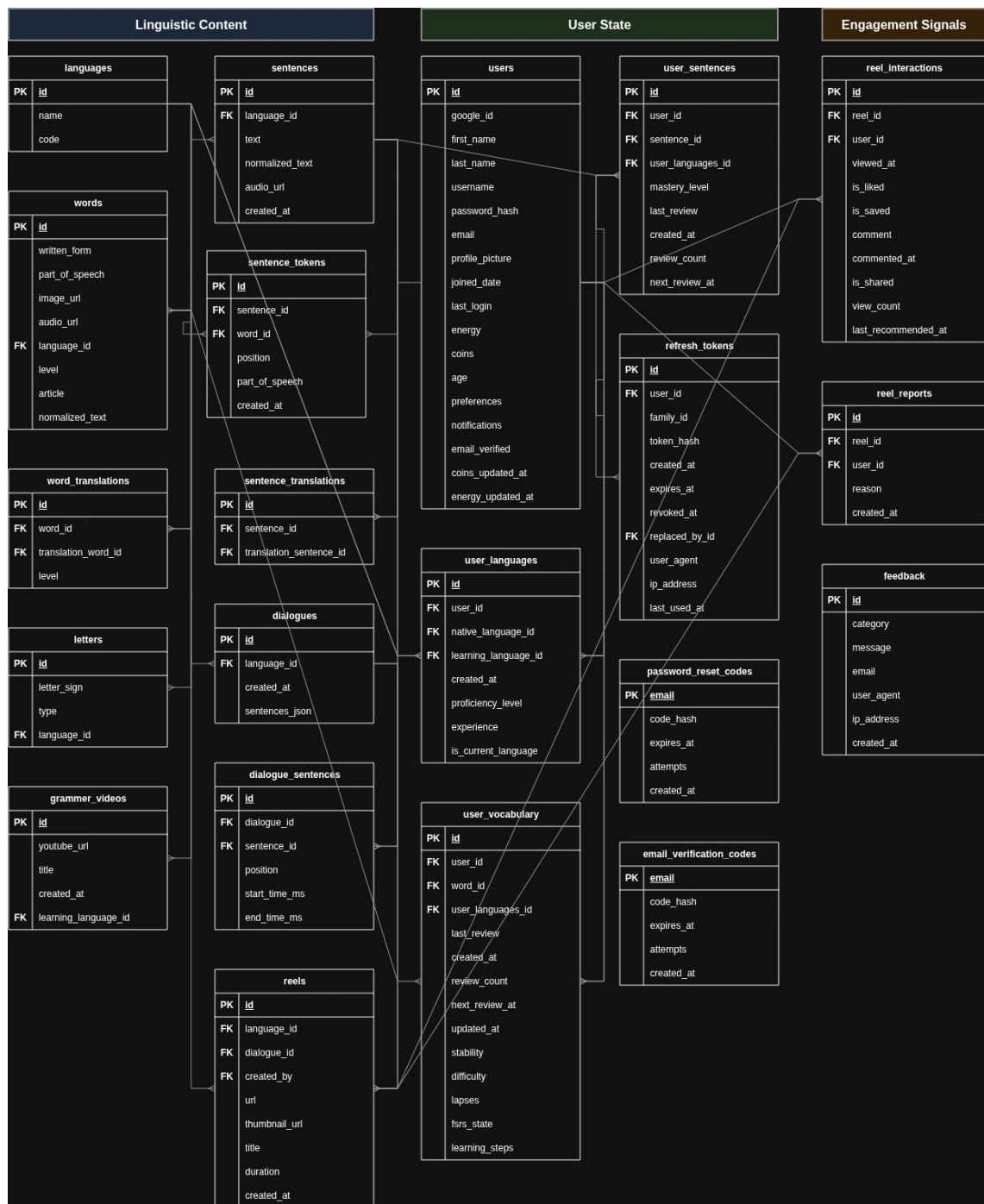


Figure C.4: Entity-relationship diagram of the 21-table schema, grouped into the three axes of Section C.4: Linguistic Content, User State, and Engagement Signals.

per-language (`sentence_translations`) annotation, `dialogues` that group sentences into a single reel’s transcript, and `reels` themselves, which reference a dialogue and their creator but otherwise carry only media metadata (URL, thumbnail, duration). A reel’s own row never records who it is comprehensible *to*: comprehensibility is only ever computed relative to a specific learner’s vocabulary at query time (Section C.6.1), not stored as a property of the reel itself, even though the reel row does reference the single user who created it.

C.4.2 User State

`users` carries authentication and profile fields alongside the two progression currencies from Section C.7.1, `coins` and `energy`. `user_languages` records one row per (user, native language, learning language) triple, including a `proficiency_level`, a per-language experience counter, and an `is_current_language` flag, since a learner may study more than one language but only one is active at a time. `user_vocabulary` is the learner model proper: one row per (user, word) pair, carrying the full FSRs memory state reviewed in Section B.2.4.4 directly as columns — `stability`, `difficulty`, `lapses`, `fsrs_state`, `learning_steps`, `review_count`, and `next_review_at` — so that “is this word due” is a plain column comparison rather than a value recomputed from a review history log on every read.

C.4.3 Engagement Signals

`reel_interactions` records one row per (user, reel) pair, accumulating likes, saves, shares, and comments as boolean/text columns rather than a separate event-log table, plus the two book-keeping fields Stage 1 and Stage 3 depend on directly: a `view_count` distinguishing a genuine repeat view from the first, and a `last_recommended_at` timestamp used to exclude a reel already shown to this learner within the cooldown window (Section C.6.1). `reel_reports` and `feedback` complete this axis, capturing moderation and product-feedback signals that do not feed the recommendation engine but share the same “signal generated by a learner about a piece of content” shape.

C.5 Onboarding and Learner-Model Initialisation

A new learner is a cold start on two counts: the vocabulary model of FR1 is empty, and there is no engagement history for Stage 3 to rank by (Section C.6). Onboarding is the design’s answer to both. It is a sequence of five short screens shown before any account exists: a welcome, the choice of native and learning language, a self-declared proficiency level, the reminder settings, and finally the learner’s age and preferred topics.

The level is declared, not measured. Each option is worded as something the learner can do, from “learning the language for the first time” to “I have mastered the language completely”, and maps onto the CEFR scale from N (a complete beginner) through A1 to C2. A placement test would measure more accurately, but it would place a test in the first minutes of use, when the attrition described in Chapter A is highest. The design accepts a less exact starting point instead, on the understanding that the memory model corrects itself as the learner’s reviews accumulate.

The declared level initialises the vocabulary model. Every word below that level is assumed known and is entered as an already-learned card, given a longer interval the further the word lies below the declared level (Section D.2.2 gives the mechanics). This assumption directly affects Stage 1: a learner who overstates their level starts with a vocabulary that is too large and is shown reels that are harder than intended, until lapses in review pull the model back. A complete beginner has no lower level to seed from, and is instead given a handful of everyday starter words as new cards; this is currently implemented for Farsi.

Because onboarding runs before an account exists, a learner can reach the reels feed and the games immediately, and their progress stays on the device. When they later sign in, they choose whether to merge the device’s progress into the account or to overwrite it with the account’s saved progress. Age and topic preferences are collected during onboarding for personalisation, but the recommender does not yet use them; Section E.2.2 evaluates this gap and Chapter F lists it as future work.

C.6 Recommendation Engine Design

The engine’s goal is to select, from the pool of reels in the learner’s target language, those that are simultaneously comprehensible, due for review, and matched to that learner’s own demonstrated interests. It does so as a three-stage pipeline over a candidate pool scoped to the learner’s language

pair: a hard filter, followed by two re-ranking passes over the filter’s survivors.

C.6.1 Stage 1: Comprehensibility Filtering

Every candidate reel is first checked against two independent conditions, either of which removes it from consideration outright. The first is recency: a reel already shown to this learner within a cooldown window is excluded, tracked via a `last_recommended_at` timestamp rather than an actual-viewing signal, since a genuine view is only confirmed after 2 seconds of continuous playback and would otherwise let an unwatched, scrolled-past reel block itself from ever being shown again. The second is the comprehensibility threshold operationalising Section B.2.2’s $i + 1$ boundary directly:

$$\text{Coverage}(U, R) = \frac{|V_R \cap V_U|}{|V_R|} \geq 0.98 \quad (\text{C.1})$$

where V_R is the unique set of word ids in reel R ’s transcript and V_U is the set of word ids anywhere in learner U ’s tracked vocabulary — membership only, independent of each word’s FSRS state, since comprehension depends on a word being recognisable at all, not on how due it is for review. This is the content-based filtering pattern reviewed in Section B.4.1: a reel’s own lexical features scored against a profile built from this one learner’s vocabulary, with no reference to any other learner. Every reel surviving both conditions carries its computed coverage forward as a `comprehensibility_percentage`, displayed to the learner alongside the reel itself.

C.6.2 Stage 2: Spaced-Repetition Prioritisation

Because FSRS runs on-device (Section C.3.1), the recommendation engine does not compute due words itself: the frontend supplies an ordered list of word ids currently due for review, oldest-due-first, with each request. Among Stage 1’s survivors, a reel containing at least one of these words is preferred over one that does not — re-ranking the survivor set rather than filtering it further, so that a learner is never shown fewer reels merely because none of them happen to contain a due word. This is the mechanism that closes the gap identified in Section B.3.1.2: watching a reel that happens to contain a due word is what triggers the in-context review step of Section C.7.2, giving passive viewing the scheduled, active-recall reinforcement it otherwise lacks. Once a page of reels is selected, due words are assigned back to specific reels on a first-due, first-assigned basis, so that a page carrying several due words can surface a distinct review prompt for each, rather than only ever reviewing the single most-overdue word.

C.6.3 Stage 3: Content-Based Personalisation

Within equal Stage 2 priority, survivors are further ranked by similarity to the learner’s own engagement history — a second content-based pass, not collaborative filtering (Section B.4.2), since no other learner’s data enters the computation. Each reel in the candidate pool is represented as a TF-IDF vector over its word-id bag (Section C.3.3 motivates the choice of scikit-learn here on engineering, not only theoretical, grounds); the learner’s profile vector is the weighted *sum* (not an average — there is no normalisation by total weight or interaction count) of the vectors of reels they have previously engaged with, weighted by the strength of that engagement (a comment or a save contributing more than a bare view), and candidates are ranked by cosine similarity to that profile. A learner with no engagement history yet simply has no profile vector, and this stage contributes no signal for them, rather than failing or substituting an arbitrary default.

C.6.4 Combining the Three Stages: A Cascade, Not a Weighted Sum

The three stages are combined as a *cascade* in Burke’s sense (Section B.4.2), not a weighted linear combination: Stage 1 is a hard pass/fail filter, and Stages 2 and 3 together form a single sort key, (has-due-word, content-similarity), applied to Stage 1’s survivors with a random tiebreak. Stage 2’s binary flag is always the primary key and Stage 3’s similarity only breaks ties within it, so a reel containing a due word is never outranked by one that merely resembles the learner’s past engagement more closely. Both signals degrade gracefully to a constant when they have nothing to work with — no word currently due, or no engagement history yet to build a profile from — which turns that half of the sort key into a no-op rather than a failure; when neither stage has any signal at all, the ranking degrades cleanly to the same random selection a brand-new learner would have received before either signal existed.

C.7 Gamification Design

Section B.3.2.1 distinguished decorative gamification — points and streaks layered onto an unchanged drill — from game-based learning, in which the reward layer is structural. *Glosy*’s progression system is deliberately scoped to the former role only: a light extrinsic-motivation layer sitting on top of the two games and the reel-review loop, which are themselves where the game-based-learning argument of Chapter B is made.

C.7.1 Progression Currencies: Coins, Energy, and Experience

Three progression stats are tracked per user (or, for experience, per learning language): coins, energy, and experience. Of the three, only coins currently have a complete reward loop: a learner earns coins for rating a spaced-repetition review prompt after watching a reel (Section C.7.2) and for winning a round of either game. The coin amounts are not yet unified into a single reward schedule, however — the review prompt and a Word of Wonders win both award a flat 20 coins, while Wordle awards 5 coins for a win and 1 for a loss — a discrepancy worth reconciling into a single schedule rather than a deliberate design choice.

Energy and experience, by contrast, are present in the data model, synced between client and server, and displayed in the profile and header UI, but have no gameplay loop attached to them yet: no screen in the application currently spends energy to start a round, and no action currently increments experience. The schema and configuration already anticipate more than is implemented — `users` carries an `energy_updated_at` timestamp alongside `coins_updated_at`, and the backend defines `MAX_ENERGY` and `ENERGY_REGEN_INTERVAL` environment variables — but no code path on either the client or the server currently reads them to compute a regenerated value, and the stored default for a new user’s energy (100) does not match the validated cap those settings imply (5). These two currencies should be read as scaffolding for a future energy-gated play loop, not as an oversight discovered late; Chapter F returns to this as a direction for future work.

C.7.2 Design Principle: Every Game Round Doubles as an FSRS Review

Both vocabulary-driven games and the reel-review prompt share one design rule: the word being tested is always drawn from the learner’s own tracked vocabulary, and preferentially from the words FSRS currently marks as due, rather than from the full dictionary or an arbitrary in-game selection. This is the direct operationalisation of Section B.2.5’s testing effect and the gap identified in Section B.3.1.2: a game round is not merely themed around vocabulary, it *is* the productive-retrieval review step that closes that gap, with the game’s win condition and the FSRS review coinciding rather than sitting side by side.

The two games apply this rule differently, however, rather than sharing one uniform mechanism. Word of Wonders only ever treats the practice word as a graded FSRS review, surfacing the learner-facing Forgot/Hard/Good/Easy review prompt on its finish screen, when that word actually came from the due-word queue — finding the board’s practice word when nothing was due is not graded

at all. The Wordle-style game does not use that self-graded prompt in any case: finding the secret word within the attempt limit is always recorded as a successful review (a fixed **GOOD** rating) regardless of whether the word was genuinely due or a random tracked fallback, and running out of attempts leaves the word's FSRS state untouched rather than recording a failed review. Word of Wonders' gate exists precisely to avoid the corruption a uniform rule would otherwise risk — grading a word that was never due would inject a review event that should not exist — while Wordle accepts that risk in exchange for a simpler, ungraded win/loss outcome; Section D.5.1 and Section D.5.2 detail each mechanism concretely.

C.8 Vocabulary-Driven Games

Both games realise the word-reconstruction format motivated in Section B.3.2.3: the player recalls and produces a word letter-by-letter rather than recognising it among distractors, and in both games that word is drawn from the learner's own tracked vocabulary under the rule of Section C.7.2. Each is also bounded in scope for the same microlearning reason as the 90-second reel cap of Section C.2: Wordle limits a round to six guesses, and Word of Wonders caps both the number of words placed on a board and the size of the shared letter pool a learner must scan (Section D.5.2), so neither game's single round can grow into a long-form session at odds with Section B.2.3's account of bounded working memory.

C.8.1 Wordle-Style Game

The learner has six attempts to guess a five-letter secret word, with each guess coloured letter-by-letter as correct, present-elsewhere, or absent, following the now-familiar Wordle mechanic. What distinguishes it from a generic port of that mechanic is that it is genuinely multilingual rather than English-only with translated labels: each of the three supported languages (English, Greek, Farsi) has its own letter-validity pattern restricting guesses to that language's script, its own on-screen keyboard layout, and, for Farsi, a right-to-left board and keyboard rather than a mirrored English one. This matters pedagogically as well as cosmetically: a Greek or Farsi learner is guessing within their target language's own orthography, not a Latin-script approximation of it.

C.8.2 Word of Wonders (Crossword)

Word of Wonders places the practice word — the due word, or a random tracked fallback (Section C.7.2) — on a grid alongside several other words from the same learner’s tracked vocabulary, each intersecting the others at a shared letter, in the style of a crossword. Where Wordle exercises one word per round, Word of Wonders’ board therefore reinforces several previously-introduced words simultaneously around the single word under FSRS review, at the cost of a puzzle that takes visibly longer to solve; the concrete placement algorithm, and the letter-budget constraint that keeps that trade-off bounded, are detailed in Section D.5.2. A secondary “extra words” mechanic rewards the learner for continuing to explore the revealed letters after the placed words are found — any additional valid word spelt out by the board’s letters counts towards a bonus, paid out in batches — extending engagement with the same board past the point where the due-word review is technically already complete.

C.9 Design Decisions and Trade-offs

Two decisions cut across the components described above and shape much of the architecture: where a learner’s state lives, and where the memory model runs. Both follow from the requirements of Section C.1, and each has a cost that the design accepts on purpose.

C.9.1 Offline-First Client State

The device, not the server, holds the working copy of a learner’s state. Tracked vocabulary, saved sentences, progress, and settings are stored on the device and used from there, so reviewing words and playing the games need no connection (NFR2). Each change is recorded as it happens in a change-set of inserts, updates, and deletes, and the change-set is sent to the server in batches whenever a connection is available. The server keeps a copy so that the learner’s state survives a reinstall or a new phone, but it does not drive the application. Section D.3.3 describes the mechanism.

Two consequences follow. A change made offline reaches the server only at the next synchronisation, so the server’s copy can lag behind the device. And the server applies the changes a device sends without versioning them or resolving conflicts, so if a learner studied on two devices at once, the later synchronisation would overwrite the fields written by the earlier one. The design there-

fore assumes that a learner mostly studies on one device. The alternative, a server that is the single source of truth, would need a connection for every review, and it would fail in the places where, as Chapter A notes, learners with limited access study.

C.9.2 Memory State on the Device

The FSRS scheduler runs on the device (Section C.3.1). The memory state of a word is therefore updated the moment the learner answers, with no round trip, and a word is due when its stored review time has passed, a comparison made on the device. The server stores the resulting state in the columns of `user_vocabulary` (Section C.4.2) but never computes it. The recommendation engine depends on this state without owning it: with each request the client sends the ordered list of due words (Section C.6.2). This is what lets Stage 2 work for a learner in guest mode, who has no account and therefore no vocabulary on the server. The same request serves guests and registered learners, and the reels service never has to store or compute a learner's schedule.

This has two costs. Logic that has to agree on both sides is written twice: the seeding of a new learner's vocabulary from the declared level exists once in the client and once in the backend, in TypeScript and in JavaScript, and the two must give the same result (Section C.5). And scheduling depends on the device's clock, so a device with a wrong clock produces wrong review times. The design accepts both, because computing the schedule on the server would put a network request in the path of every review, which is incompatible with practising offline.

Chapter D

Implementation Details

D.1 Database Schema

The schema comprises 21 tables in a single PostgreSQL 18 database (Section C.3.4), organised below along the same three axes introduced in Section C.4.

D.1.1 Linguistic Content Tables

`languages`, `words` and `word_translations`, and `letters` hold the base vocabulary. A sentence’s text lives in `sentences`, its per-word annotation in `sentence_tokens` (word id, position, part of speech), and its cross-language equivalents in `sentence_translations`. `dialogues` groups an ordered sequence of sentences via `dialogue_sentences` (which also carries each sentence’s `start_time_ms/end_time_ms` within the video), and `reels` references exactly one dialogue alongside its media URL, thumbnail, duration, and creator. Rather than re-joining `sentences`, `sentence_tokens`, and `sentence_translations` on every read, `dialogues` additionally carries a denormalised `sentences_json` column: a pre-computed snapshot of the full dialogue, tokens and all-language translations included, written once when a reel’s subtitles are created or edited and read directly thereafter (Section D.4.2).

D.1.2 User State Tables

`users` holds authentication fields and the two implemented-and-scaffolded progression currencies of Section C.7.1 (`coins`, `energy`, each with its own `_updated_at` timestamp). `user_languages` records one row per language pair a user studies, with its own `proficiency_level` and `experience`. `user_vocabulary` is the FSRS learner model described in Section C.4.2, carrying `stability`, `difficulty`, `lapses`, `fsrs_state`, and `learning_steps` directly as columns. `user_sentences` tracks per-sentence progress on a separate, coarser scheme: a `mastery_level` between 1 and 6, alongside its own `review_count` and `next_review_at`, distinct from `user_vocabulary`'s continuous FSRS state. `refresh_tokens` (family-scoped, for the rotation scheme of Section D.2.1), `password_reset_codes`, and `email_verification_codes` round out this axis with authentication-adjacent state.

D.1.3 Engagement Signal Tables

`reel_interactions` carries one row per (user, reel) pair: boolean `is_liked/is_saved/is_shared` flags, an optional comment, a `view_count`, and the `last_recommended_at` timestamp Stage 1 filters on (Section C.6.1). `reel_reports` and `feedback` record moderation reports and free-form product feedback respectively; neither feeds the recommendation engine, but both share this axis's shape of a learner-generated signal attached to a specific piece of content.

D.2 Backend API

The API is documented with Swagger/OpenAPI, generated from JSDoc annotations on each route and served at `/swagger`. Every request body is validated against a Joi schema before it reaches a controller (Section C.3.2), and large, uniform list responses — the dictionary word list and a learner's vocabulary state — are serialised in a columnar `{columns, rows}` shape rather than one JSON object per row, avoiding repeated field names across thousands of otherwise identically-shaped entries.

D.2.1 Authentication and Authorization

D.2.1.1 JWT Access/Refresh Token Rotation

Login and registration issue a short-lived access token and a longer-lived refresh token. Rather than storing refresh tokens verbatim, the `refresh_tokens` table stores only a hash of each one, grouped into rotation *families* by a `family_id`. Every use of `POST /refresh` runs a single atomic operation: it locks the presented token's row (`SELECT . . . FOR UPDATE`, inside a transaction, so two concurrent refresh attempts against the same token cannot race each other), and then branches on that row's state:

- if the row is already `revoked`, the presented token has already been rotated away once before — a replay of an old token — so every active token in that family is revoked immediately and the request is rejected;
- if the row is unrevoked but past `expires_at`, the family is revoked and the request is rejected;
- otherwise, a new token row is inserted into the same family, copying the original `expires_at` verbatim so a session's absolute lifetime never resets on activity, the presented row is marked revoked with a `replaced_by_id` pointer to the new one, and the new access/refresh pair is returned.

This gives the family reuse-detection property standard to refresh-token rotation: a stolen refresh token is only usable once before the legitimate client's next refresh attempt discovers the token has already moved on and revokes the whole family, rather than the theft going undetected indefinitely.

D.2.1.2 Google OAuth 2.0

`verifyGoogleToken` middleware validates a client-supplied Google ID token against the OAuth client id registered for the requesting platform, using `google-auth-library's OAuth2Client.verifyIdToken`, before either the login or registration controller runs. On success it attaches the decoded profile (Google id, email, name, picture) to the request; the downstream controller then finds-or-creates a user keyed by that Google id exactly as it would for a password-based account, so both sign-in paths converge on the same JWT issuance logic described above.

D.2.1.3 Password Reset and Request Limiting

Password recovery uses the same hashed-code pattern as registration. The forgot-password endpoint always returns the same generic response, whether or not the address belongs to an account, so it cannot be used to discover which emails are registered. Only the email that is sent differs: a reset code for a password account, a “not registered” notice, or a pointer to Google sign-in for an account that has no password. A second request within 60 seconds is absorbed without sending another email. The code is stored only as a hash, expires after a fixed time, and is rejected once a limited number of wrong attempts has been made. A successful reset writes the new password hash and revokes every refresh token of that user, so a session held by someone else does not survive the change.

Each sensitive endpoint also has its own rate limiter over a 15-minute window. The tightest limits sit on the endpoints that send email or accept guessable codes: 5 requests for code sending and for forgot-password, 10 for login, code verification and password reset, and 30 for token refresh.

D.2.2 Registration and Vocabulary Initialisation

Registration requires a verified email address: a code sent to the address must be confirmed against a hashed, expiring, attempt-limited pending record before any user row is created, and the pending record is deleted on success so the code cannot be reused. The username is derived from the email’s local part, with two disambiguating fallback variations (an underscore or dot plus a numeric suffix) tried in sequence if the plain form is already taken.

D.2.2.1 Proficiency-Based Vocabulary Auto-Seeding

Immediately after a new language pair is created, if the learner declared a proficiency level above the lowest, `userVocabularyModel.addByProficiencyLevel` seeds `user_vocabulary` with every word at a level below their stated one, on the assumption that a self-declared B1 learner already knows their A1 and A2 vocabulary without having reviewed it inside *Glosy*. Seeded words are inserted directly into FSRS’s Review state rather than New, with an initial stability drawn — uniformly at random per word, via a single SQL CASE expression keyed on each word’s own level — from a range that widens with distance below the target level (up to 7 days for the level immediately below, 20 for two levels below, 60 for three or more): a word closer to the learner’s stated level is assumed more recently and less thoroughly reinforced than

one several levels more elementary. Randomising per word rather than assigning one flat value per level matters operationally as much as pedagogically: without it, an entire level's worth of seeded words — potentially thousands, for a learner who declares a high starting level — would all share the same `next_review_at` and become due for review on the exact same day.

D.2.3 Offline Synchronisation

A single `POST /sync` endpoint reconciles everything the frontend's offline-first client (Section D.3.3) has accumulated since its last successful sync: updated `user_progress` (coins, energy), and `vocabulary_changes/sentence_changes`, each split into `inserts`, `updates`, and `deletes` keyed by word or sentence id.

D.2.3.1 Change-Set Ordering: Deletes, Updates, Inserts

Within each change-set, the three operations are applied in a fixed order — deletes, then updates, then inserts — rather than in whatever order the client happened to record them. This ordering is what makes the endpoint correct in the presence of a delete-then-recreate on the same word or sentence id during one offline session (a learner removing a tracked word and later re-adding it before ever reconnecting): applying the delete first and the insert last guarantees the row exists with the client's latest state once the request completes, regardless of the order the two actions occurred in on-device. Vocabulary and sentence updates are applied as set-based `UPDATE . . . FROM (VALUES . . . )` statements, one per chunk of 500 rows, inside a single transaction, so a large offline session costs a handful of database round trips rather than one per word, and a failure part-way leaves no partially applied batch. Only rows that still exist are updated, so an id deleted in the meantime is skipped rather than recreated, and a field the client did not send keeps its stored value while an explicitly sent `0` or `null` is written. The transaction covers only this update step: the endpoint's other steps (user progress, deletes, inserts) are separate statements, so the sync as a whole is not atomic.

D.2.4 Dictionary and Letters Modules

The dictionary endpoints serve a language's full word list, optionally joined with translations into a second language, keyed by ISO language code and validated against a fixed three-language enum (English, Farsi, Greek) hard-coded into the request schema, rather than a live lookup against

whichever languages happen to be present in the database at the time. The letters module serves each language’s alphabet as its own lightweight dictionary — vowels and consonants for Farsi split further into short/long forms, per the RTL and script-specific handling already noted for the Wordle-style game (Section C.8.1) — so that alphabet drills can reuse the same word-list UI components as ordinary vocabulary instead of a bespoke screen.

D.2.5 Account Deletion and Personal Data

The application keeps little personal data: an email address, name and username, a password hash, an age, the topic preferences chosen at onboarding, the language pair, the vocabulary state, and the content the user creates. The privacy policy, served from the backend’s static directory, states that the application uses no location data, contacts, advertising or analytics SDKs, and that it is intended for users aged 16 and over, which is the lowest age the onboarding screen accepts. Two lookups bypass the servers altogether: the device fetches word definitions directly from the English Wiktionary and loads grammar videos directly from YouTube (Section D.3.5), so those providers see the request rather than *Glosy*.

A user can delete their account permanently from within the application. The backend deletes the `users` row, and foreign keys declared `ON DELETE CASCADE` remove the user’s languages, vocabulary, sentences, refresh tokens, reels, reel interactions, and reports. The controller then removes the user’s media from object storage, under the `reels/` and `profile_pictures/` prefixes for that user’s id. The two steps are not atomic: if the storage cleanup fails after the row has been deleted, the request reports an error although the account is gone, and the files remain in storage. On the client, the `useDeleteAccount` hook changes local state only after the server confirms the deletion. It then stops pending synchronisation, clears the stored tokens and all persisted data, resets every context and the onboarding flag, and returns the learner to the welcome screen. If the request fails, local data is left untouched.

D.3 Frontend Architecture

D.3.1 Navigation and Screen Structure

Expo Router maps the file system directly onto the navigation tree. The five tabs from Section C.2 — Home, Reels, Create, Practice, Profile — sit under a single (`tabs`) route group, each inde-

pendently stackable (Create, for instance, pushes a details and a subtitle-sync screen on top of its own tab without affecting the other four); onboarding, settings, and the two full-screen games each live under their own top-level route group outside the tab bar, since none of them should show tab chrome while active.

D.3.2 State Management: Context and Hooks

Vocabulary state is split across two parallel structures rather than one: `userVocabulary` itself (keyed by word id, holding each word's FSRS fields) and a separate `vocabularyChanges` object recording only the inserts/updates/deletes made since the last successful sync. A single dispatch function updates both together for an ordinary edit, through two separate reducers, so the change-set is always an accurate diff rather than something reconstructed by re-comparing full snapshots at sync time. Two call sites deliberately bypass the change-set half of this: `bulkAddVocabulary`, used for onboarding auto-fill, since that seed mirrors what the backend's own proficiency-based auto-seed (Section D.2.2) already creates server-side and so is not itself a change to push back; and `reviewWord`, which computes a word's next FSRS state on-device via `scheduleReview` (Section C.3.1) and dispatches it as an ordinary `UPDATE` — a graded spaced-repetition review is not treated as a special kind of state change requiring its own sync path, it is simply an update to the same fields any manual edit would touch.

D.3.3 Offline-First Strategy

Every piece of state that must survive an app restart — vocabulary, its pending change-set, sentences, user progress — is held through a shared `usePersistedState` hook backed by `AsyncStorage`, each exposing an `isLoading` flag so dependent logic (most importantly, sync) can wait for disk hydration to finish before deciding anything is dirty. `useBackendSync` generalises over an arbitrary list of such change-sets and pushes them to `POST /sync` (Section D.2.3) under three independent triggers: a five-minute interval while changes are pending, the instant `isOnline` transitions from false to true, and the app moving to the background — covering the periodic case, the reconnection case, and the “about to be suspended, don't lose this” case respectively, rather than relying on any one of them alone. Dirtiness itself is tracked precisely rather than optimistically: a change to `coins` or `energy` is only flagged when its actual value diverges from the last value known to be reflected server-side, so that a reference-only update to `userProgress` — switching the active language, or applying a fresh snapshot the server itself just returned — does

not mark anything dirty and trigger a needless sync. A failed sync leaves its change-sets marked dirty for the next trigger to retry, surfaces an error banner once rather than on every retry, and treats an HTTP 401 as “this session is over,” clearing the dirty flag rather than retrying a request that will never succeed until the learner logs in again.

D.3.4 Practice Reminders

The daily practice reminder is a local notification scheduled on the device with `expo-notifications`. The server takes no part in it: no push token is stored, and the reminder works offline. The settings (enabled, hour, minute) are persisted through the same `usePersistedState` mechanism as the rest of the client state (Section D.3.3), and a single effect in `ReminderContext` keeps the operating system’s schedule in step with them. Whenever a setting or the learner’s first name changes, it cancels the notification registered under a fixed identifier and schedules a new daily trigger, or only cancels when reminders are off. Because the identifier is fixed, rescheduling on every launch cannot create duplicates. The message addresses the learner by first name, and tapping it opens the Reels tab, including when the application is launched cold from the notification.

Permission is requested only when the learner opts in, either with the “Remind me” button on the onboarding screen or by switching the reminder on in Settings. If the operating system refuses, the switch stays off and the learner is told to enable notifications in the device settings. On Android the reminder uses its own notification channel. The notifications module is loaded lazily and skipped in Expo Go on Android, where SDK 53 removed push support and importing the module would otherwise throw.

One trade-off was accepted deliberately. On Android 12 and later, a scheduled notification is delivered through an inexact alarm unless the application holds the `SCHEDULE_EXACT_ALARM` permission, so a reminder can arrive a few minutes late (about two minutes in testing). The application does not request that permission: a nudge to practise is not time-critical, an exact-alarm permission adds a request flow for the learner, and Google Play restricts the permission to applications whose main function is alarms or calendars.

D.3.5 Grammar Video Library

Grammar lies outside the vocabulary model (Section A.4), but the application points learners to a curated library of external YouTube lessons. A `grammar_videos` table stores each lesson's YouTube link, an optional title, and its learning language, and a rate-limited `GET /videos/:language_code` endpoint returns the lessons for one language in the order they were added. The Videos screen, opened from an icon in the header of the home tab, lists them as cards with the YouTube thumbnail. Tapping a card plays the lesson in place with `react-native-youtube-iframe`, one at a time. The video id is extracted from any of the link shapes YouTube produces (`watch`, `youtu.be`, `embed`, `shorts`), so links can be stored exactly as shared.

Because a video's owner can disable embedding, a player error marks that card, and the next tap opens the video in YouTube through an in-app browser instead. Titles left empty when the library was seeded are fetched at display time from YouTube's `oEmbed` metadata. The library currently holds 45 lessons, all for Farsi, arranged from the alphabet and pronunciation through pronouns and the verb “to be” to noun plurals, and the other languages show an empty list. Watching a lesson is not recorded and does not feed the vocabulary or recommendation model.

D.4 Reels Service

D.4.1 Service Architecture

The `reels` microservice (`reels-service/`) follows a conventional FastAPI layering: `routers` parse and validate the HTTP layer only, delegating immediately to a `services` class holding the actual business logic (`ReelService` for reading/recommending reels, `ReelCreationService` for writing them), which in turn operates on SQLAlchemy `models` through Pydantic `schemas` at the boundary. `GET /reels` is deliberately dual-mode: it decodes the same JWT the Node backend issues (Section D.2.1) if one is present, but does not require it, so an unauthenticated request still gets a working reels feed via `get_random_reels` while an authenticated one gets the full recommendation pipeline via `get_personalized_reels` — there is no separate “guest mode” flag anywhere in the request, only the presence or absence of a decodable token.

D.4.2 Reel Creation and Dialogue Enrichment

Creating a reel with its subtitles is one all-or-nothing transaction: a `dialogues` row, one `sentences` row per subtitle line (reusing an existing row for identical text in that language rather than duplicating it), one `dialogue_sentences` row per line carrying its millisecond-precise start/end time, and any translation lines linked in via `sentence_translations`. A line's position in the client-supplied array, not any position value the client sends, determines its stored `position`, so a malformed or out-of-order request can never violate the dialogue's position ordering. Tokenising each sentence into `sentence_tokens` — which requires the lemmatisation libraries Section C.3.3 deliberately keeps out of this service's live request path — does not happen at creation time; a newly-created reel's dialogue therefore has no per-word token data until a separate offline script backfills it. What is computed immediately, in the same transaction, is the `sentences_json` denormalisation of Section D.1.1: one aggregating SQL query rebuilds the dialogue's full JSON snapshot — text, every language's translations, and whatever tokens already exist — once, at write time, rather than reassembling it from four joined tables on every subsequent read. Editing a published reel's subtitles (`PUT /reel/{id}/dialogue`) reuses the identical insert logic over the new lines, after first clearing the dialogue's existing `dialogue_sentences` links, and recomputes the same JSON snapshot.

D.4.3 Recommendation Engine Implementation

D.4.3.1 Stage 1: Candidate Loading and Comprehensibility Scoring

The candidate pool for a request is every reel in the learner's target language, loaded with its language, creator, and dialogue eager-loaded in one query — the catalog is small enough at present that scoring the whole pool in application code is acceptable, a scaling assumption Section D.4.4 returns to. Two SQL-backed exclusions run first: reels already recommended to this learner within the cooldown window (a `reel_interactions.last_recommended_at` lookup), and, separately, whether the learner has a `user_languages` row for this pair at all — if not, the request falls through entirely to the same unauthenticated `get_random_reels` path a guest gets, since there is no tracked vocabulary yet to filter on. For each surviving candidate, its unique word-id set is extracted from the cached `sentences_json` in Python (no query, since it is already in memory) and intersected against the learner's known-word set, also fetched once per request; a candidate passes only once that ratio reaches `COMPREHENSIBILITY_THRESHOLD` (0.98, Sec-

tion C.6.1). A reel with no tokens yet — one whose dialogue has not been through the offline tokenisation backfill — cannot be measured against this threshold at all, and is excluded rather than assumed comprehensible.

D.4.3.2 Stage 2: Frontend-Supplied Due-Word Partitioning

The engine never computes which words are due itself: `due_word_ids` arrives as a comma-separated, oldest-first query parameter, populated from the on-device FSRS state of Section D.3.3. Stage 1’s survivors are partitioned by simple set membership — does this reel’s word-id set intersect the due-word set at all — and that boolean, not the number or identity of due words matched, is the primary key of the sort described next. Once a page is selected, due words are assigned back to specific reels oldest-first, each to the first still-unclaimed reel in that page containing it, so that a single response can carry a distinct review prompt for more than one due word rather than only ever the single most-overdue one.

D.4.3.3 Stage 3: TF-IDF Profile Vectors and Cosine Similarity

Every candidate’s word-id bag is vectorised together, over the whole pool rather than per-request, so that the TF-IDF weighting’s inverse-document-frequency term reflects the full catalog and not just one learner’s survivors: `DictVectorizer` turns each reel’s `{word_id: count}` mapping into a sparse count matrix, and `TfidfTransformer` weights it. The learner’s profile vector is the sum of the TF-IDF rows for every reel they have previously interacted with, each row scaled by that interaction’s weight (a flat 0.2 for a bare view, stacked with 1.0 for a like, 1.5 for a save, 1.0 for a share, and 1.5 for a comment, so a single heavily-engaged reel can outweigh several passive views); a learner with no interaction history yet simply has no profile vector, and `cosine_similarity` is skipped rather than computed against a zero vector.

Stages 2 and 3 are combined as a single sort key over Stage 1’s survivors, `(has_due_word, similarity)`, applied after a random shuffle so equally-ranked candidates do not always resolve in the same order: this is Python’s ordinary stable tuple sort, not a weighted formula (Section C.6.4). When a request has neither a due-word list nor an existing profile vector to work with, both components of the key are constant across every candidate, and the stable sort degrades to the pre-shuffle random order — the same random selection `get_random_reels` would have produced.

D.4.4 Personalisation Implementation Status

Three simplifications are explicit, commented design decisions in the code rather than oversights discovered while writing this thesis. First, the candidate pool is loaded and scored in full on every request; the service’s own comment on this notes it directly: the catalog is small enough for now that this is acceptable, and it should move to a SQL-side computation once the reel catalog grows large enough for per-request, in-memory TF-IDF fitting to become a bottleneck. Second, there is no collaborative-filtering component anywhere in the engine, for the cold-start reason already given in Section B.4.2: a cross-learner interaction matrix dense enough to be useful does not yet exist for a newly-launched application, so Stage 3 substitutes a second content-based pass over the learner’s own history rather than attempting a collaborative signal on too little data. A third, narrower limitation follows directly from Section D.4.2: a reel is only ever comprehensibility-scored once its dialogue has been tokenised, so a newly-uploaded reel is effectively invisible to the personalised feed until the offline backfill script has processed it, a manual step rather than an automatic part of publishing.

D.5 Game Modules

D.5.1 Wordle-Style Game Implementation

The secret word is drawn exclusively from the learner’s own `userVocabulary` (Section C.7.2), never the full dictionary: due words are collected via `getDueWords` (Section C.3.1) and one is chosen at random when at least one exists, falling back to a random tracked word otherwise. Guesses and the secret word are compared through the same `normalizeWord` canonicalisation used throughout the app — stripping Greek diacritics and Farsi harakat and uppercasing Latin script — so that a learner is not penalised for a guess that differs from the stored form only in diacritics their keyboard does not easily produce. Word length (5), attempt count (6), the permitted letter pattern, and the on-screen keyboard layout are all resolved per-language from `getWordleConfig` (Section C.8.1) rather than hard-coded, and the board and keyboard render right-to-left when the active language’s config marks it as such. The due-word and tracked-word sets are read through refs rather than directly inside the round-initialisation callback, specifically so that a mid-round `reviewWord` call — which updates `userVocabulary`, and so would otherwise change those sets’ identity — does not itself retrigger initialisation and silently restart the round before the learner reaches the collect step. Unlike Word of Wonders (Section D.5.2), Wordle does not use the self-

graded Forgot/Hard/Good/Easy prompt at all: guessing the secret word within the attempt limit always records a review with a fixed **Good** rating, whether or not that word was genuinely due, and running out of attempts records no review at all rather than a failing one. Coins are awarded through the same `userProgress` update path regardless of outcome (5 for a win, 1 for a loss).

D.5.2 Word of Wonders: Procedural Crossword Generation

D.5.2.1 Word Placement and Intersection Search

A level is built on an 8×8 grid, starting from the practice word and greedily placing further words from the learner’s tracked vocabulary wherever they intersect an already-placed word at a shared letter. A candidate placement is rejected if it would overlap a perpendicular word at a non-shared cell, or touch an existing word edge-to-edge without actually crossing it, which is what keeps every placed word legible as a separate word rather than fusing into an unintended run of letters. Algorithm 1 summarises the core loop, run until either every candidate word has been tried or `MAX_WORDS` (12) placements have been made.

D.5.2.2 Letter-Budget and Word-Priority Heuristics

The player picks letters for each word from a single shared pool of physical letter tiles shown alongside the board, rather than a full keyboard, so that pool must stay small regardless of how many words end up on the board. `MAX_LETTERS_LENGTH` (8) bounds it: a candidate word is only placed if the *multiset union* of its letters with the letters already in use — not the sum of both counts — stays within that budget, so a word reusing letters already on the board is cheaper to add than one introducing entirely new letters. Candidate words are tried in priority order for exactly this reason (Algorithm 2): ranked first by how many letters they share with the pool already in use, and second (as a tiebreak) by how few new letters they would add, so the generator exhausts low-cost, letter-reusing words before it is forced to spend budget on words that would grow the pool.

D.5.2.3 Due-Word Injection as the Practice Word

The practice word is the one word in a Word of Wonders level whose discovery is treated as an `FSRS` review, and it is only ever marked as such — surfacing the self-graded rating prompt of

Algorithm 1 Word of Wonders board generation (per level)

```

1: words  $\leftarrow$  learner's tracked vocabulary, filtered to valid length and script for the target language
2: practice  $\leftarrow$  due word if one exists and survives the filter above, else a random word from words
3: place practice horizontally at a random row on the empty board
4: sort remaining words by the priority heuristic of Algorithm 2 (descending)
5: for each word in sorted words, while placed count < MAX_WORDS do
6:   if placing word would exceed the letter budget (Section D.5.2, next) then
7:     skip word
8:   end if
9:   for each already-placed word p do
10:    for each shared-letter intersection between p and word do
11:      if the resulting placement is valid (in bounds, no illegal overlap) then
12:        place word at that intersection; break out of both loops
13:      end if
14:    end for
15:  end for
16: end for
17: trim the board to the bounding box of all placed cells

```

Algorithm 2 Word priority for placement ordering

```

1: shared, added  $\leftarrow$  0, 0
2: for each letter  $\ell$  appearing in word or already in the pool do
3:   before  $\leftarrow$  pool's current count of  $\ell$ 
4:   after  $\leftarrow$  max(before, word's count of  $\ell$ )
5:   if after > before then
6:     added  $\leftarrow$  added + (after - before) {word needs more of  $\ell$  than the pool has}
7:   else
8:     shared  $\leftarrow$  shared + before {pool already covers  $\ell$ ; credit its full existing count}
9:   end if
10: end for
11: return (shared, -added) {sort descending: more shared letters first, then fewer added letters}

```

Section C.7.2 on the finish screen — when it was actually drawn from the due-word queue rather than substituted at random because nothing was due. Every other word placed on the board is practice in the ordinary sense (the learner has tracked it before and may re-encounter it here) without being graded as a spaced-repetition event, keeping FSRS state changes limited to words genuinely scheduled for review rather than every word that happens to appear on a given board. This due-word gate is specific to Word of Wonders; the Wordle-style game (Section D.5.1) grades every completed round regardless of due-word status, rather than gating on it the same way.

D.6 Deployment and Distribution

The backend and the reels service each run on their own host, behind their public addresses. All configuration is read from environment variables rather than from code: database credentials, token secrets and lifetimes, the Google client ids for each platform, the storage credentials, and the mail settings. The backend refuses to start if its storage configuration is incomplete.

Media never passes through the API servers. To upload a reel or a profile picture, the client asks the API for a presigned upload URL, valid for 15 minutes, for a key under that user's own prefix. It sends the file directly to the Cloudflare R2 bucket, and the server checks that the object exists before it writes the file's URL to the database. This keeps large video bodies off the small hosts, consistent with the memory limit of the reels service (Section C.3.3).

The mobile application is built with Expo Application Services using three profiles: a development client for internal use, a preview profile that produces an Android APK, and a production profile that produces an Android app bundle for the internal track of Google Play. The Android APKs are also published as versioned releases on GitHub, while the iOS build is currently available only through Expo.

Chapter E

Evaluation & Results

E.1 Evaluation Approach

As acknowledged in Chapter F, *Glosy* has not yet been evaluated in a controlled study with human learners. This is the main disadvantage of the present work: none of the results in this chapter can speak to actual vocabulary gain, retention, or engagement, since no learner has yet used the system under observation. In the absence of that evidence, this chapter instead evaluates the system’s own algorithmic components directly, against real seeded content, to establish whether each behaves as its design in Chapter C intends and where it does not — a necessary but not sufficient substitute for pedagogical evidence, and one that Chapter F returns to among its directions for future work. Each section below targets one component in isolation. The crossword generator and the Stage 2 ranking are run as the real, unmodified implementation from Chapter D; the Stage 1 comprehensibility filter is recomputed from the seeded data by a script that mirrors its formula.

E.1.1 Research Questions and Evidence

Table E.1 shows which research question each experiment speaks to and what it leaves untested. Every research question has at least one component evaluated, and none of them can be answered in terms of learning outcomes.

Table E.1: Evidence gathered for each research question, and what remains untested.

RQ	Evaluated	Where	Not evaluated
RQ1	Word-level linkage of reel transcripts to the vocabulary model, exercised through Stage 1’s coverage measure	Sec. E.2.3	Whether tracked words are actually learned
RQ2	Comprehensibility pass rates per level; due-word prioritisation on a guest’s first page	Sec. E.2.3, E.2.4	Content-based ranking (Stage 3); relevance as the learner sees it
RQ3	Crossword generator: robustness, board richness, generation time	Sec. E.2.1	Wordle-style game; effectiveness as spaced-repetition practice
All	Batched offline synchronisation and token rotation, as supporting infrastructure	Sec. E.3	Registration, password reset, account deletion

E.1.2 Data and Reproducibility

Every experiment runs on the system’s own seeded content rather than on synthetic stand-ins, but not all of that content can be shared. The crossword benchmark samples from the three seeded dictionaries (13,373 English, 13,722 Greek, and 14,315 Farsi entries). These dictionaries are part of the repository, as are the harness, its raw per-trial results, and the aggregated summary, so the crossword figures can be regenerated, or the 4,200 trials repeated, from the repository alone. The recommender experiments use the Farsi reel catalogue of 56 reels, of which 50 have tokenised transcripts and can therefore be scored (Section E.2.3). That catalogue is read from a full export of the production database, which contains user data and is deliberately kept out of the repository. The recommender scripts under `scripts/evaluation/` are published, but their input is not, so these two experiments cannot be repeated from the repository alone. They can be repeated on any database export that contains the reels and their tokenised transcripts, and their results are reported here as measured.

E.2 Component-Level Algorithmic Evaluation

E.2.1 Word of Wonders: Grid-Generation Benchmark

The crossword placement algorithm of Section D.5.2 (Algorithms 1 and 2) is bounded by three constants: an 8×8 board, a practice-word length of at most 8 letters, a shared letter-budget of 8 distinct-letter slots (`MAX_LETTERS_LENGTH`), and a cap of 12 placed words (`MAX_WORDS`). Because the algorithm’s behaviour depends on how large and how diverse a candidate word pool it is asked to search, and a learner’s tracked vocabulary grows from a handful of words at first use to potentially thousands after months of use, the two questions this evaluation asks are: does the generator remain robust as tracked vocabulary grows, and how do the two quantities that matter to a learner — how rich the resulting puzzle is, and how long they wait for it — scale with that growth.

E.2.1.1 Method

A benchmark harness (`scripts/evaluation/wow_generator/` in the project repository) imports `LevelGenerator.js` unmodified and drives it with word pools sampled directly from the seeded per-language dictionaries of Section C.4 (13,373 English, 13,722 Greek, and 14,315 Farsi entries). For each of the three supported languages and seven simulated tracked-vocabulary sizes (10, 100, 500, 1,000, 3,000, 6,000, and 10,000 words — the last comfortably exceeding what any real learner’s tracked vocabulary reaches in practice, to characterise the algorithm’s behaviour at the extreme rather than only the typical case), 200 repetitions drew an independent random subset of that size from the full dictionary, selected one of its words uniformly at random as the FSRS-due practice word (mirroring `GameLoader.js`’s own unfiltered selection — Section C.7.2), and called the generator exactly as the application does. This produced 4,200 trials in total, recording the words placed, the wall-clock generation time, and whether an exception was raised.

E.2.1.2 Robustness

Across all 4,200 trials, at every vocabulary size in every language, the generator raised no exception and always returned a non-empty board with its practice word placed. This holds even at the smallest tested vocabulary (10 words), where the candidate pool for intersecting words is thin, and at the largest (10,000 words), where the priority sort and intersection search run over a considerably

larger candidate pool per call than any real session is likely to present it with.

E.2.1.3 Words Placed

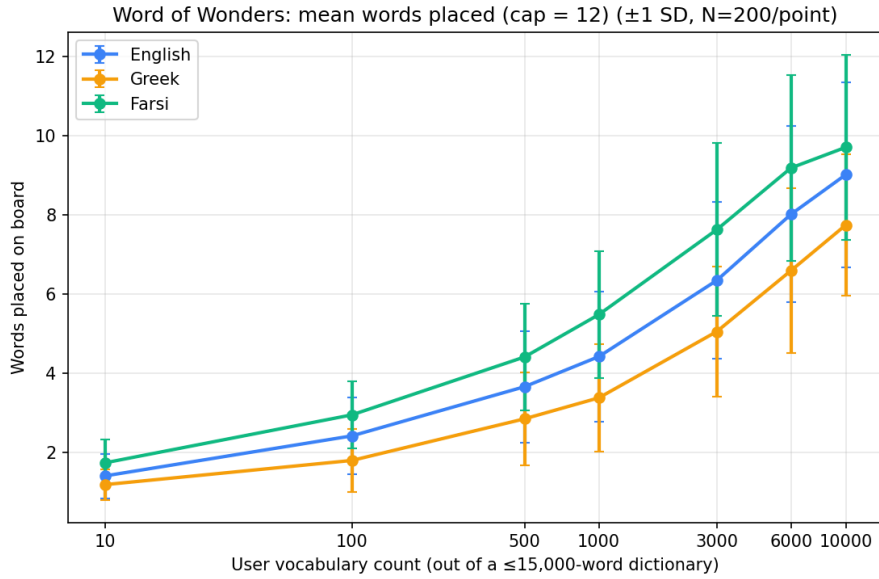


Figure E.5: Mean words placed on the board (cap = 12) as a function of simulated tracked-vocabulary size, per language. Error bars show ± 1 standard deviation over 200 repetitions per point.

Figure E.5 shows words placed rising with vocabulary size in all three languages, from a mean of 1.2–1.8 words at a 10-word vocabulary to 7.7–9.7 words at 10,000 — yet even at that extreme, the board never saturates the 12-word cap. This confirms the design argument of Section C.8.2: the binding constraint on puzzle richness is the 8-slot letter budget of `wouldExceedLetterLimit`, not the word count or the cap itself, since a bigger, more diverse candidate pool exhausts the shared letters faster than it exhausts the word cap. A second, consistent pattern across every vocabulary size is a language ordering — Farsi places the most words, English second, Greek fewest — consistent with average word length: among the words the generator accepts (three to eight letters after normalisation), the mean length is 5.1 letters for Farsi, 6.0 for English, and 6.5 for Greek, so the language with the shortest words places the most, since a longer average word consumes more of the same 8-letter budget per placement.

A direct consequence for onboarding is that a brand-new learner, whose tracked vocabulary is by

construction small, is served a sparse board of only one or two words rather than a filled one — compounding, rather than mitigating, the “steep onboarding barriers” limitation already acknowledged in Chapter F.

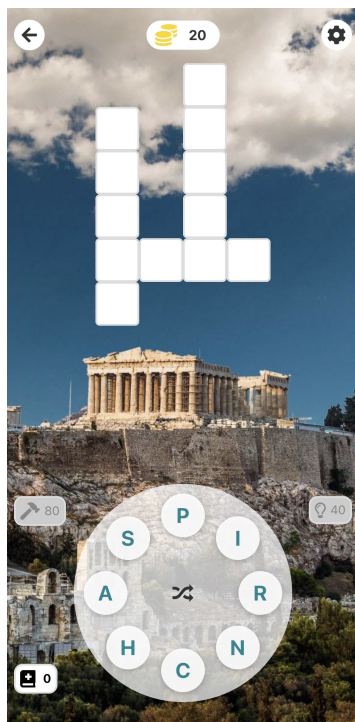
E.2.1.4 Qualitative Example: On-Device Boards Across CEFR Levels

Figure E.6 shows the same generator running on a physical device, for one learner who selected each of the six CEFR levels in turn. The pattern corroborates Figure E.5 directly, but through a different mechanism than the benchmark’s own uniform random sampling: proficiency-based vocabulary auto-seeding (Section D.2.2) inserts every word at every level *below* the learner’s declared one into `user_vocabulary` at registration, so declaring A1 — already the lowest level — seeds nothing and leaves `GameLoader.js`’s `trackedWords` pool empty of anything beyond whatever the learner has reviewed since, while declaring C2 seeds the entire A1–C1 dictionary at once. The visibly sparse A1/A2 boards and the visibly denser B2–C2 ones are therefore a real instance of the same word-count effect Section E.2.1.3 measured synthetically, produced by the application’s actual onboarding path rather than by the benchmark harness.

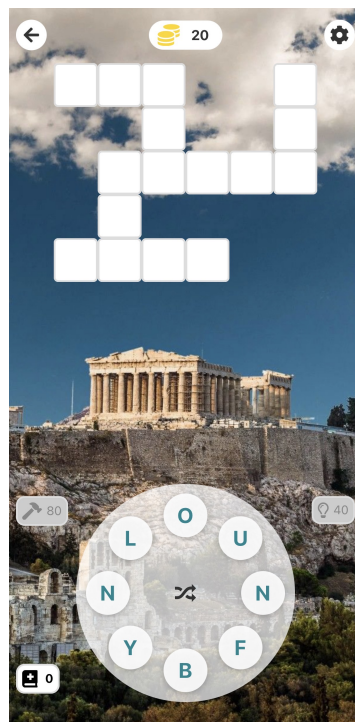
E.2.1.5 Generation Time

Figure E.7 shows generation time growing from sub-millisecond at a 10-word vocabulary to 159–192 ms at 10,000 words, with the three languages tracking each other closely throughout. The curve is visibly superlinear beyond roughly 3,000 words, consistent with `tryPlaceWord`’s per-candidate intersection search scaling with both the number of words already placed and the priority sort (Algorithm 2) running over the entire candidate pool on every call.

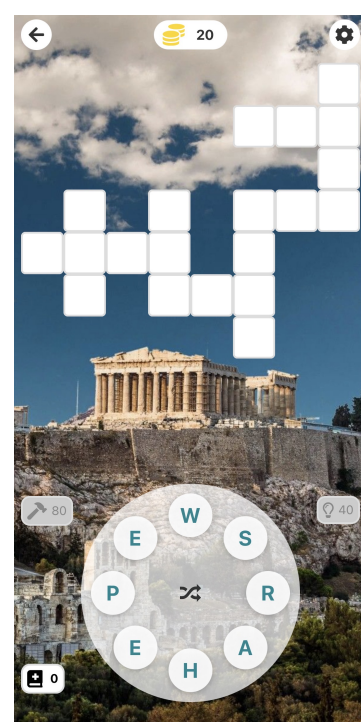
These figures were measured under Node.js/V8 on a laptop, not under Hermes on an actual mobile device, and are reported here as evidence of the algorithm’s *scaling behaviour* — no combinatorial blow-up and no failure at any tested scale — rather than as a prediction of on-device latency: JIT and garbage-collection behaviour differ enough between V8 and Hermes, and between individual phones, that an absolute millisecond figure measured on one device would not transfer to another. `GameLoader.js` currently calls the generator synchronously on the JS thread (Section D.5.2), wrapped only in a zero-delay `setTimeout` to yield a render frame for the loading spinner, not to move the computation off-thread; the superlinear growth observed here means that a learner whose tracked vocabulary eventually reaches the thousands of words could, on slower hardware, experience a visibly longer wait than the benchmark’s low-vocabulary numbers suggest. A direct



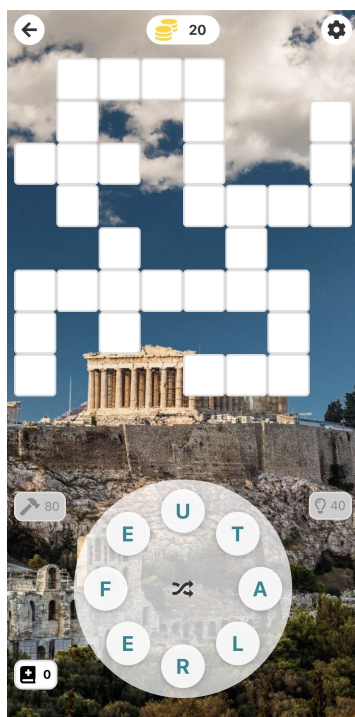
(a) A1



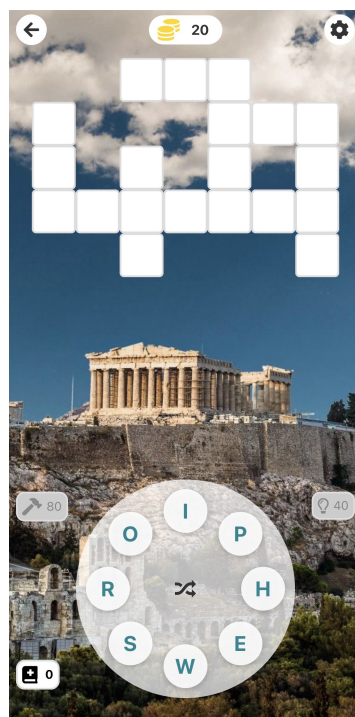
(b) A2



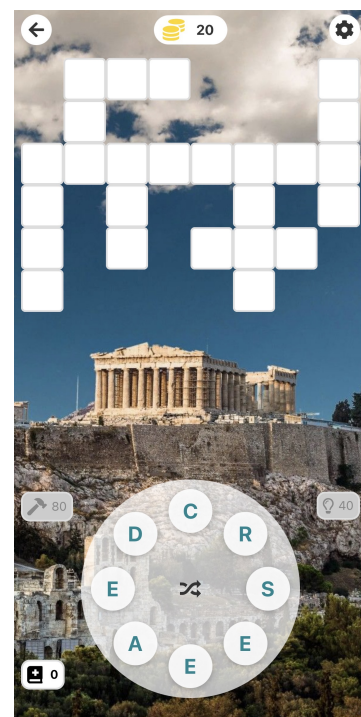
(c) B1



(d) B2



(e) C1



(f) C2

Figure E.6: Word of Wonders boards generated on a physical device for an English learner who declared each of the six CEFR levels at onboarding, all other conditions held fixed. Board density visibly grows from a bare two-word cross at A1 to a multi-armed board of several intersecting words by B2–C2.

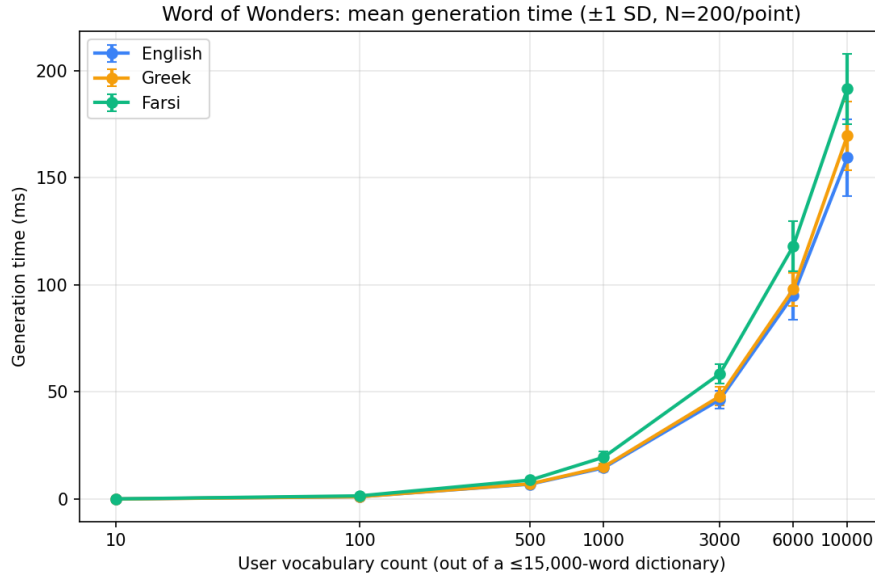


Figure E.7: Mean grid-generation wall-clock time (ms) as a function of simulated tracked-vocabulary size, per language, measured under Node.js/V8 on a development machine. Error bars show ± 1 standard deviation over 200 repetitions per point.

mitigation, left as a concrete direction rather than implemented here, is to move grid generation server-side — the reels service’s existing infrastructure (Section D.4) already performs comparable per-request computation — and push a precomputed board to the client opportunistically the next time it is online, trading a network round trip for a guaranteed, device-independent generation cost rather than an open question per phone.

E.2.2 Recommendation Engine: The Cold-Start Problem

Cold start — a recommender system having little or nothing to personalise on for a given user — is a well-known limitation of collaborative and content-based recommendation alike, not a defect specific to *Glosy* [49]. Section B.4.2 already gave this as the reason the engine has no collaborative-filtering stage at all: a cross-learner interaction matrix dense enough to be useful does not yet exist for a newly-launched application. What this section evaluates is how the same problem manifests one level down, inside Stage 3 itself, and what *Glosy* currently does and does not do about it.

Stage 3’s profile vector (Section C.6.3) is built entirely from `reel_interactions` — the weighted sum of the TF-IDF vectors of reels a learner has liked, saved, shared, or commented

on. Since every one of *Glosy*’s current users is, by definition, a user who has not yet accumulated that history, `profile_vector` is `None` for all of them today, and Section C.6.4’s graceful degradation takes over: the cascade’s sort key collapses to (has-due-word, constant 0), i.e. Stage 2 plus a random tiebreak, with Stage 3 contributing nothing. This is not a rare edge case the design merely tolerates — it is, at present, the actual ranking behaviour for the entire user base, for as long as no one has interacted with enough reels to build a profile from.

Glosy already has a partial, in-progress answer to this in its onboarding flow, one screen ahead of where the cascade could use it. The last onboarding page (`PersonalizationSlide.tsx`, Figure E.8) asks the learner to multi-select from a fixed set of eight topic chips — Movies, Sports, Anime, Make up, Cartoons, Video games, News, Politics — and to declare an age (16 upward, matching the declared Play Store audience), both required before onboarding can proceed. Both are persisted at registration: `preferences` as a comma-joined string and `age` as an integer, both columns on `users` (Section C.4). Neither, however, is read anywhere in `reel_service.py` — not in Stage 3’s profile construction, not anywhere else in `get_personalized_reels` or `get_random_reels`. The data is collected and stored; the recommendation engine simply does not look at it yet. This is a genuine gap rather than a documentation lag: the schema and the onboarding UI are already ahead of the recommendation code that would consume them.

Two concrete directions follow directly from this gap, both already anticipated by the existing structure rather than requiring a new data model: first, using the declared topic preferences as a cold-start substitute for Stage 3’s profile vector — a sensible prior for a learner who has stated an interest in, say, Sports, before they have liked a single reel, closing the gap identified in the paragraph above — currently limited by the fixed, hand-picked eight-topic taxonomy the onboarding chips offer, with automated topic generation (deriving topics from reel content rather than a static hardcoded list) as the natural next step once a large enough content library makes that worth building; and second, age-conditioned recommendation, using the already-collected `age` column as a further ranking or filtering signal once a concrete policy for it is designed. Both are carried forward as named future-work items in Chapter F.

E.2.3 Recommendation Engine: Stage 1 Comprehensibility Filter

Stage 1’s outcome (Section C.6.1) depends less on the filter itself than on the catalogue it is applied to: how many reels exist for the learner’s language, how densely each reel’s subtitles are tokenised (a reel with no tokenised word can never pass), and how many reels were recently recommended

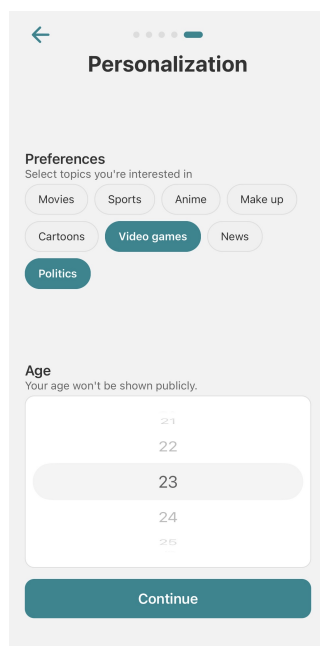


Figure E.8: The Personalization screen (`PersonalizationSlide.tsx`), the last onboarding page

to the learner, since these are excluded for a 24-hour cooldown window. The results below are therefore illustrative rather than a property of the engine alone. The catalogue currently holds only 56 Farsi reels, inserted only to show how the filter behaves rather than curated as a learning library; 50 of the 56 have at least one tokenised word, and the other six can never pass yet.

Figure E.9 shows that at the 0.98 threshold the application uses, beginners are barely served (level N: 4%; A1: 12%), the pass rate rises through A2 (36%) and B1 (56%), and it reaches only 88% even at C2. For comparison, Stage 1 was also evaluated at a looser threshold of 0.5, which saturates from A2 upwards (92%) but still passes only 8% and 18% of reels for levels N and A1. A looser filter therefore does not close the beginner gap: that needs easier content, not a lower threshold, and the application keeps the 0.98 threshold.

E.2.4 Recommendation Engine: Stage 2 Due-Word Prioritisation

Stage 2 (Section C.6.2) can only prioritise reels if the learner has words due for review. A learner who declares level N for Farsi has no lower level to seed vocabulary from, so onboarding instead adds four everyday starter words — *salām* (hello), *āb* (water), *bābā* (dad) and *chāy* (tea) — as

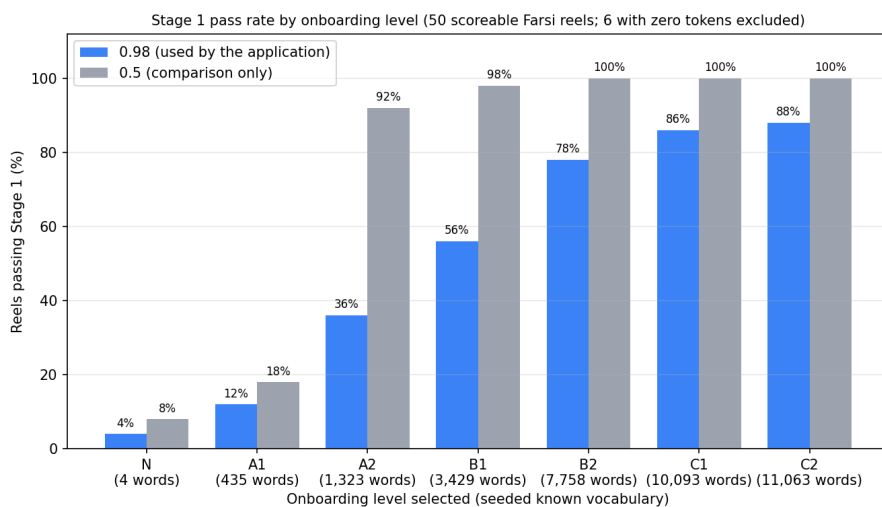


Figure E.9: Share of the 50 tokenised Farsi reels passing Stage 1, per onboarding level, at the threshold the application uses (0.98) and, for comparison only, a looser one (0.5). Each level’s known vocabulary is what registration seeds for it (Section D.2.2); level N receives only four starter words.

brand-new FSRS cards, all due immediately. The catalogue of 56 Farsi reels was assembled so that reels containing these words exist: 11 of the 56 contain at least one of them (*salām* in 5, each of the other three in 2).

The scenario tested is a new user who has not registered and opens the reels feed. The frontend forwards the four due words with the request, and for a guest the server applies Stage 2 alone, since the comprehensibility filter and the content-based ranking both need an account. Running the reels service’s own ranking and review-word assignment code on the real catalogue, over 2,000 repetitions, every reel on the first page of ten contains a due word, against 1.96 of ten for a page requested without due words, and all four words receive a review prompt on every page, each attached to a different reel. The mechanism therefore behaves as designed, but only because the catalogue was built around these four words: it holds just 11 such reels, so a second page can carry at most one more. As with Stage 1, what limits the feature is the size of the catalogue, not the ranking logic.

E.2.5 Recommendation Engine: Stage 3 Content-Based Ranking

Stage 3 (Section C.6.3) is not tested here. It is built entirely on scikit-learn's TF-IDF weighting and cosine similarity, an established library implementing a long-standing retrieval technique, so re-testing those components would evaluate the library rather than *Glosy*. What remains specific to the system, its dependence on an interaction history that no user has yet accumulated, is discussed in Section E.2.2.

E.3 Backend Correctness Testing

The algorithmic experiments above do not cover the code that keeps a learner's data consistent between the device and the server. The backend therefore has an automated test suite, centred on the synchronisation endpoint (Section D.2.3). The endpoint applies the vocabulary and sentence updates in chunks of 500 rows, one set-based statement per chunk, inside a single transaction. Because this code silently holds learner state, a defect in it would go unnoticed, so the tests check its behaviour directly: the meaning of each field, the transactional guarantees, and agreement with a simple reference implementation. The suite also covers refresh-token rotation and logout. It has 33 test cases in four files and is run with `npm test`.

The tests are integration tests, not unit tests with mocks. They run under Jest against a real PostgreSQL database: the endpoint tests call the real Express application through Supertest, and the batching tests call the update functions directly. Each file creates a throwaway user with its own language, words, and sentences, and deletes it afterwards; the foreign keys' `ON DELETE CASCADE` removes everything that depends on it.

Field semantics. Twenty tests cover the batched vocabulary and sentence updates, 12 and 8 respectively. They check the field semantics that the client depends on. Fields that were not sent keep their stored value, an explicit `0` is written rather than treated as "unchanged", and an explicit `null` clears the nullable columns. An id with no row is skipped and not created, so a word deleted in the meantime is not resurrected. An empty batch returns at once without touching the database. Two keys that name the same id, such as `"1"` and `"01"`, are merged with the later entry winning per field, and only the rows of the given language entry are touched.

Batching and failure behaviour. A batch of 1,100 rows, which needs three chunks, must be applied as exactly one `BEGIN`, three `UPDATE` statements, and one `COMMIT`; the test counts them by recording every statement sent over the connection. Failure must be all-or-nothing: when an

invalid value in the last chunk violates a `NOT NULL` constraint after the earlier chunks have already run, the table must end up exactly as it was. Finally, after 15 consecutive failing calls, more than the pool's ten connections, the pool must still be usable with no connection left checked out.

Agreement with a reference implementation. The strongest check is a differential test. A straightforward row-by-row implementation, with one `UPDATE` per row, is kept in the test file as a reference. For both vocabulary and sentences, a randomised payload of 200 rows is applied to identically seeded rows, once by the reference and once by the batched code. The payload mixes omitted fields, zeros, and nulls, and comes from a fixed-seed generator so that a failure can be reproduced. The test requires the same final table and the same returned rows, so the batching is shown to change how the update is executed and not its result.

The endpoint. Three tests drive the sync endpoint itself. A mixed payload that deletes, updates, and inserts must apply the deletes first, then the updates, then the inserts. It includes an update for a word deleted in the same sync, which must be skipped, and a word deleted and inserted again, which must come back fresh. The response must have the expected keys, with updates keyed by id and carrying every field. An updates-only payload returns the updated rows keyed by id. A payload with an invalid update must answer with a server error and leave no part of the batch in the database.

Sessions. Ten further tests cover the session endpoints. For refresh tokens they check that a valid token is rotated, that replaying a used token is detected and revokes the whole family including the token that replaced it, that a token past its absolute lifetime is rejected as expired, and that a garbage token is rejected differently from a reused one. They also check that two device sessions of one user are independent and that rotation does not extend the family's expiry. For logout they check that only the presented token's family is revoked, so other devices stay signed in, and that logging out twice or without a token is harmless.

The suite has clear limits. It covers the server only: the frontend has no substantive automated tests. The atomicity it verifies is that of the update step, and the sync as a whole is still not atomic (Section D.2.3). It does not test registration, password reset, account deletion, or the rate limiters. It also does not exercise the case that the row lock in the refresh endpoint exists to protect, two refresh requests for the same token arriving at the same moment.

E.4 Discussion and Threats to Validity

E.4.1 What the Results Show

Taken together, the experiments answer a narrower question than the research questions ask: whether each component does what its design says. On that question the answer is largely yes. The crossword generator did not fail in 4,200 trials, and its boards grow richer as the learner's vocabulary grows (RQ3). Stage 2 puts a due word in every reel on a guest's first page and gives every due word a review prompt (RQ2). The batched synchronisation and the token rotation are covered by 33 automated tests, all of which pass, including a comparison against a row-by-row reference implementation (Section E.3).

The results also share a pattern: where a component falls short, the limit is the amount of data, not the logic. At the 0.98 threshold Stage 1 passes only 4% of reels for a level-N learner and 12% for A1, and a looser threshold does not help. Stage 2 works because the catalogue was built around four starter words and holds only 11 reels that contain them. The crossword gives a new learner a board of one or two words because their tracked vocabulary is small. In each case more content, or a larger vocabulary, would improve the outcome without any change to the algorithm. This is a design finding as well as a limitation: the next investment should be content, above all easier reels for beginners, rather than a more elaborate ranking.

E.4.2 Threats to Validity

Five limitations qualify these results.

Construct validity. Pass rates, words placed, and generation times are properties of the algorithms. None of them measures whether a learner understands a reel or remembers a word, so the results show that the mechanisms work, not that learners benefit from them.

Simulated learners. The benchmark draws tracked vocabularies uniformly from the dictionary, whereas a real learner's vocabulary is structured by level. The Stage 1 experiment assumes that a learner knows exactly the words that registration seeds for the declared level (Section C.5). Real learners will differ, so the absolute numbers should be read as indicative.

A small, curated catalogue. The recommender experiments use 50 tokenised Farsi reels chosen by the maintainer to demonstrate the filter (Section F.2), and the Stage 2 result depends on the catalogue having been built around the four starter words. Results for English or Greek, or for a

larger and less curated library, could differ.

Measurement conditions. Generation times come from a laptop running Node.js, not from phones running Hermes, and each figure comes from a single machine. They are evidence of how the algorithm scales, not of latency on a device.

Limited reproducibility. The recommender results cannot be independently repeated from the repository, because the reel catalogue they were measured on is not published (Section E.1.2). A reader can inspect the scripts but must either take the measured pass rates as reported or repeat the experiments on their own reels.

Chapter F

Conclusions & Future work

This thesis asked whether the media that learners already consume habitually — short-form video and hypercasual games — can be turned into an effective, personalised vocabulary-learning environment (Chapter A). It answered by designing and building *Glosy*, a cross-platform mobile application in which a single word-level learner model, holding an FSRS memory state for every word a learner tracks, drives both the selection of reels and the words of every game round. This chapter states what the work contributes, what it does not yet establish, and where it should go next.

F.1 Contributions

The contributions of this thesis are the following.

1. **A word-level learner model shared by content and play.** Every word a learner tracks carries a Free Spaced Repetition Scheduler (FSRS) state, updated on the device after each graded review, so the application works offline and synchronises when a connection returns (Sections D.3.3 and D.2.3). Reel transcripts are tokenised and linked to the same word identifiers, so that a word tapped in a subtitle, a word due for review, and a word in a game round are one and the same record.
2. **A machine-checkable operationalisation of comprehensible input.** The recommender keeps a reel only if the learner already knows at least 98% of its words, turning Krashen’s qualitative $i + 1$ boundary into a computable filter grounded in the lexical-coverage result of

Hu and Nation (Section B.2.2). It is the first stage of a three-stage cascade in Burke’s sense: a hard comprehensibility filter, then a spaced-repetition stage that ranks reels containing due words first, then a content-based stage that breaks ties by TF-IDF cosine similarity to the learner’s engagement history. Each stage degrades to a constant when it has no signal, so the cascade never fails for a new learner (Section C.6.4).

3. **Games whose rounds are spaced-repetition reviews.** A multilingual Wordle-style game, with per-language alphabets and a right-to-left board and keyboard for Farsi, and Word of Wonders, a crossword generated on the device from the learner’s own vocabulary under a letter budget, both draw their words from the tracked vocabulary and preferentially from the words FSRS marks as due (Section C.8). The win condition of a round and the FSRS review coincide, rather than the game merely being themed around vocabulary.
4. **A content pipeline that keeps the request path light.** Learners can create reels in the application, with timestamped subtitles and translations, and edit them after publishing. Tokenisation and lemmatisation, which need spaCy, Stanza, or Hazm, run in an offline backfill rather than in the reels service, so that the service fits a 512 MB host and depends only on scikit-learn’s lightweight vectorisers at request time (Sections C.3.3 and D.4.2).
5. **A working, multilingual prototype.** *Glosy* is a deployed three-service system (React Native frontend, Node.js backend, FastAPI reels service) over one PostgreSQL schema, released as an Android application. It supports English, Greek, and Farsi, including a right-to-left script, in any native/learning combination, with a dictionary of roughly 41,000 words tagged by CEFR level.
6. **A component-level evaluation that locates the real bottlenecks.** Since no study with learners exists, Chapter E evaluated the algorithms directly. Across 4,200 trials the crossword generator never failed, and the richness of its boards was bounded by the letter budget rather than the word cap. The recommender’s Stage 1 and Stage 2 behaved as designed, but what limited them was the size and tokenisation of the catalogue, not the ranking logic.

F.1.1 Answers to the Research Questions

RQ1 asked how a mobile application can integrate authentic short-form video with a word-level vocabulary model. The answer given here is architectural: reels carry timestamped, tokenised, translated transcripts whose tokens are the rows of the learner’s own vocabulary table, so that

watching a reel can both read from and write to the learner model. **RQ2** asked how a recommendation engine should select reels that are both comprehensible and lexically challenging. The answer is the cascade above, whose ordering — comprehensibility first, due words second, similarity last — is justified by the memory and input theory of Chapter B, and whose behaviour on a real, if small, catalogue was tested in Chapter E. **RQ3** asked to what extent hypercasual mechanics, parameterised by the learner’s vocabulary state, can serve as effective spaced-repetition exercises. This thesis shows that they can be built and that the generator is robust at every tested vocabulary size, but the word *effective* cannot be answered without learners: whether these rounds improve retention is precisely the question the missing user study would address (Section F.2).

F.2 Limitations

The work presented in this thesis has several limitations that must be acknowledged honestly.

F.2.1 Evidence and Content

No empirical user study. The pedagogical effectiveness of the system has not been evaluated in a controlled experiment. Chapter E shows that the components behave as designed, not that learners gain or retain vocabulary, or that they stay engaged. Design parameters taken from the literature — the 98% coverage threshold and the default FSRS parameters — have likewise not been validated on *Glosy*’s own learners.

Limited content library. The current content library is limited in scope and diversity. The catalogue evaluated in Chapter E holds 56 Farsi reels, added by a maintainer to demonstrate the filter rather than curated as a learning library. *Glosy* faces the cold-start problem of any new application [49]: with no user base yet, no learners have uploaded reels, so the library stays small, and the recommender in turn has few reels to choose among and no interaction history to personalise from. The dictionary is larger, but it was assembled from published word and frequency lists with the help of a large language model, and its entries and translations have not been independently audited in this work.

F.2.2 Language Processing

Tokenisation is incomplete for Farsi. Some languages differ in register between speech and writing, and attach affixes to words to express what other languages express with separate words. Farsi does both: its colloquial forms differ systematically from the written standard, and a suffix can turn *mādar* (mother) into *mādaram* (my mother). English and Greek raise this problem to a much smaller degree. The Farsi tokeniser used here, Hazm, does not recognise every word form of this kind, even when the corresponding word is in the `words` table, despite the informal-text normalisation and possessive-suffix expansion added to it (Section D.4.2). A token that is not recognised is not stored, and this has two consequences. First, the word appears in the subtitles as plain text: it cannot be tapped, translated, or added to the learner’s vocabulary. Second, it is left out of the reel’s word set, so Stage 1’s coverage is measured over the words the pipeline recognised, not over every word spoken in the reel, and the displayed comprehensibility percentage can overstate how much of the reel the learner understands. How often this occurs has not been quantified here.

No word-sense disambiguation. When a lemma matches more than one dictionary entry, the pipeline links the entry with the lowest identifier, without regard to part of speech or meaning, so a token can be linked to the wrong homograph.

Offline annotation of new reels. A newly created reel has no token data until the backfill script has run, and until then it is invisible to the personalised feed (Section D.4.4). Publishing is therefore not yet self-service.

F.2.3 Recommendation and Progression

Stage 3 has fewer signals than designed. The profile vector weights views, likes, saves, shares, and comments (Section C.6.3), but only views, likes, and saves are currently recorded. The share button opens the operating system’s share sheet without recording anything, and the comment sheet is an interface mock-up without a backend. Together with the absence of any interaction history (Section E.2.2), this means Stage 3 currently contributes nothing to the ranking of any learner.

Vocabulary is seeded from a self-declared level. Registration assumes that a learner who declares B1 already knows every A1 and A2 word (Section D.2.2). A learner who overestimates their level starts with an inflated known-word set, so Stage 1 lets through reels that are too hard for them; the seeded FSRS states are also assumptions, not observations. Stage 1 also counts any tracked word as known, whatever its FSRS retrievability, which is deliberate (Section C.6.1) but means a word

that has since been forgotten still counts towards coverage.

Incomplete progression system. Only coins have a complete reward loop, and their amounts differ between the games (Section C.7.1). Energy and experience are stored and displayed, but nothing spends or increments them. The Wordle-style game grades a win as a fixed **GOOD** rating and records nothing on a loss (Section C.7.2), a coarser signal than the self-graded prompt of Word of Wonders.

Scaling of the current implementation. The recommender scores the whole candidate pool in memory on every request (Section D.4.4). The crossword generator runs on the JavaScript thread, and its cost grows superlinearly with vocabulary size; it was measured under Node.js on a laptop, not on a phone (Section E.2.1.5). None of these matters at the present scale, but each would at a larger one.

F.2.4 Onboarding and Social Scaffolding

Steep onboarding barriers. Like LingQ, *Glosy* builds on authentic material, and such an interface poses steep onboarding barriers for novice learners [24]. The evaluation shows this directly: at the 98% threshold only 4% of the tokenised reels pass for a learner who declares no prior knowledge and 12% at A1, and a new learner's Word of Wonders board holds only one or two words (Sections E.2.3 and E.2.1.3). A looser threshold does not close this gap; easier content does.

Lack of social scaffolding. The system lacks mechanisms for social scaffolding through peer or native-speaker interaction. Learners can like, save, and share a reel, but they cannot yet ask a question about it or receive a correction, so the sociocultural side of the zone of proximal development (Section B.2.1) is approximated only by the lexical threshold and not by human support. The planned remedy is the reel's comment section: the comment sheet is already part of the interface, but it is not yet connected to a backend. Once it is, a learner who meets an idiom or a cultural reference in a reel can ask about it directly beneath that reel, where the creator or another learner can answer, which is the content-attached scaffolding that Section B.1.2 found missing from existing platforms.

Three languages, each hand-configured. Only English, Greek, and Farsi are supported, and each language has its own hand-written configuration: a letter pattern and keyboard for Wordle, a lemmatiser for tokenisation, and its own entry in the language enumeration of the API and in the onboarding options. The Wordle-style game also fixes the secret word at five letters.

F.3 Future Work

The following directions are identified as high-value:

- **Empirical evaluation:** A randomised controlled study measuring vocabulary gain, retention at one week and one month, and learner engagement metrics, comparing *Glosy* against a no-game control condition and against Duolingo. It would also test whether the 98% threshold is the right value for a video-based learner and whether the game rounds improve on passive review.
- **Automated content annotation pipeline:** A pipeline that ingests a raw video URL, transcribes speech using a large language model, aligns the transcript to video timestamps, tokenises and POS-tags the output, maps tokens to the word database, and produces a fully annotated reel record—reducing the per-reel curation cost from hours to minutes, and removing the manual backfill step that currently keeps new reels out of the personalised feed.
- **Manual tokenisation while creating a reel:** Letting the creator correct the tokenisation of a subtitle line inside the reel-creation flow, by marking which dictionary word each unrecognised word belongs to. This would recover the words that Farsi’s colloquial forms and attached suffixes cause the tokeniser to miss (Section F.2), make them tappable, and keep them in the coverage calculation; the creator’s corrections could also serve as training data for a better tokeniser.
- **Coverage over all running words:** Storing unrecognised tokens as such, instead of dropping them, so that Stage 1 can measure coverage over every word of a transcript and report a comprehensibility percentage that does not depend on the tokeniser’s recall.
- **Cold-start mitigation via onboarding preferences:** Wiring the topic preferences and age already collected by onboarding’s `PersonalizationSlide.tsx` (Section E.2.2) into Stage 3 as a cold-start substitute for the interaction-based profile vector, replacing its current fixed, hand-picked eight-topic taxonomy with automated topic generation derived from reel content once the library is large enough to support it.
- **Age-conditioned recommendation:** Using the `age` column, already collected at onboarding but currently unread by the recommendation engine, as a further ranking or filtering signal once a concrete policy for it is designed.

- **Comment section and collaborative learning features:** Connecting the reel comment section to a backend, so that learners can ask questions and receive answers from the creator or from other learners beneath the reel they concern, which is the main remedy for the missing social scaffolding (Section F.2). Beyond it, shared vocabulary challenges, leaderboards scoped to friend groups, and discussion attached to individual words would extend the social dimension of Vygotsky's theory. Persisting comments and shares would also give Stage 3 the two signals it is currently designed for but never receives.
- **Multi-language expansion:** The schema is language-agnostic (language is a foreign key throughout), so a new language needs mainly content ingestion. It also needs a small amount of per-language configuration — a lemmatiser, a Wordle letter pattern and keyboard layout, and an entry in the language enumeration — which could be moved into the database so that adding a language stops requiring a release.
- **Server-side game generation and recommendation scoring:** Generating Word of Wonders boards on the server and pushing them to the device while it is online, so that generation cost no longer depends on the phone (Section E.2.1.5), and moving Stage 3's TF-IDF computation into SQL or a cache once the catalogue outgrows in-memory scoring.
- **A complete progression loop:** Unifying the coin schedule across the games and either attaching a gameplay loop to energy and experience, as the schema and configuration already anticipate (Section C.7.1), or removing them.
- **Placement instead of self-declaration:** Replacing the self-declared proficiency level with a short adaptive placement test, so that the seeded vocabulary and its FSRS states start from observed rather than assumed knowledge.
- **Exercises attached to each reel:** Offering a short set of drill-based exercises after a reel, built from its own transcript and target words: matching words to translations, sorting the words of a sentence into order, selecting the right word or meaning, and speaking a line aloud. Unlike the decontextualised drills criticised in the literature review (Chapter B), these would use the sentence the learner has just watched. They would also give an objective correct-or-incorrect result on which to rate the review, in place of the current self-graded prompt (Section C.7.2), and the speaking exercise would remove the lack of pronunciation practice noted in Section A.4. This direction builds on the choice of FSRS as the memory model, made in part because every item carries its own difficulty, a value from 1 to 10 that

is revised after each review (Section B.2.4.4). Giving each sentence a difficulty in the same way would let the exercises for a reel be chosen at a level that suits the learner, and would let reels themselves be recommended by the difficulty of their sentences.

- **Grammar-aware recommendation:** Extending the recommendation system from vocabulary to grammar. A grammar ontology would be built for each language, describing its constructs and how they relate, for example which construct is a prerequisite of which and at what level each is normally learned. Every sentence would then be linked, through the ontology, to the constructs it contains, and SPARQL queries over these links would find the reels whose sentences use constructs a learner is ready for or has yet to review. Grammar would then play the part in Stages 1 and 2 that words play today: a reel would be judged by how much of its grammar the learner already knows, and constructs would be scheduled for review like words. The grammar lessons of Section D.3.5 could be attached to the same ontology entries, so that a learner who meets an unfamiliar construct in a reel is pointed to the matching lesson.
- **A social platform:** Making the application a social platform as well as a learning tool, in which learners can follow creators and one another and chat directly. This builds on the comment section described above, and it needs safeguards that the current reel reports do not provide: reporting and blocking for users and messages, and an updated privacy policy, since messages are personal data that would also have to be deleted with the account (Section D.2.5).
- **An AI assistant:** A conversational assistant inside the application, in which a learner can ask grammatical and cultural questions about what they are learning. Given the reel or sentence being watched and the learner's tracked vocabulary as context, it could explain in words the learner already knows. Its answers would need care, because language models tend to make more mistakes in languages with less training data, such as Farsi. Grounding them in the grammar ontology above could help, and unlike the offline-first core of the application the assistant would need a connection.

F.4 Concluding Remarks

Glosy shows that a single word-level memory model can connect the two places where learners already spend their attention: the videos they watch choose which words are due, and the games

they play review those same words. Whether this makes learners retain more than they would with existing applications is not yet known, and the limitations above show that a larger, better-annotated content library is the first thing to build before that can be tested. The thesis's claim is therefore a modest one: the design is coherent, implemented, and robust at the component level, and it is ready to be tested with learners.

Bibliography

- [1] Hermann Ebbinghaus. *Memory: A Contribution to Experimental Psychology*. Teachers College, Columbia University, New York, 1885. Translated by Henry A. Ruger and Clara E. Bussenius, 1913.
- [2] Jaap M. J. Murre and Joeri Dros. Replication and analysis of ebbinghaus' forgetting curve. *PLOS ONE*, 10(7):e0120644, 2015.
- [3] Zirguez. Leitner system (alternative). Wikimedia Commons, https://commons.wikimedia.org/wiki/File:Leitner_system_alternative.svg, 2012. CC0 1.0 Public Domain Dedication. Accessed: 2026-09-15.
- [4] Kent State University. Global language learning trends and statistics. Kent State University Department of Modern & Classical Language Studies Blog, August 2025. Accessed: 2026-08-26.
- [5] Mordor Intelligence. Online language learning market size & share analysis - growth trends and forecast (2026 - 2031). Technical report, Mordor Intelligence, August 2026.
- [6] TalentLMS. The talentlms 2026 annual l&d benchmark report. Technical report, TalentLMS, 2026.
- [7] Neal Mohan. The future of YouTube in 2026: Annual letter from the CEO, January 2026.
- [8] Mark Zuckerberg. Meta's third quarter 2025 earnings conference call, October 2025.
- [9] Kepios. Essential TikTok statistics, audience reach and demographics, March 2025.
- [10] Similarweb. Digital 2026: Global overview report (android app insights), October 2025.
- [11] Pew Research Center. Teens, social media and AI chatbots 2025. Technical report, Pew Research Center, December 2025.

- [12] Pew Research Center. Americans' social media use 2025. Technical report, Pew Research Center, November 2025.
- [13] Hasan Cagli. Short-form video statistics 2026 (TikTok, reels & shorts), April 2026.
- [14] Wistia, Inc. State of video report: Video marketing statistics for 2026. Technical report, Wistia, April 2026.
- [15] Vidyad. Video in business benchmark report. Technical report, Vidyad, 2024.
- [16] Oxana Zelevskaya. Hyper-casual games: definition, genre history and specifics, 2022.
- [17] Anthony Pizzo. Hypercasual and hybrid-casual video gaming: A digital leisure perspective. *Leisure Sciences*, 47:1333–1352, 05 2023.
- [18] Business Research Insights. Hyper casual gaming market size, share, growth, trends, global industry analysis, by type (android games and ios games) by application (men users and women users), regional insights and forecast from 2026 to 2035. <https://www.businessresearchinsights.com/market-reports/hyper-casual-gaming-market-103060>, 2026.
- [19] Benjamin Paaßen, Thekla Morgenroth, and Michelle Stratemeyer. What is a true gamer? the male gamer stereotype and the marginalization of women in video game culture. *Sex Roles*, 76:421–435, 2017.
- [20] Kevin Anderton. Hypercasual female gamers are taking over the industry [infographic], September 2020. Accessed: 2026-08-27.
- [21] Alyssa Celatti. Study: Millions of us seniors aged 65+ spent \$500 on gaming in the past year, 2022. Accessed: 2026-08-27.
- [22] Zichu Yang and Bowen Sun. Hyper-casual endless game based dynamic difficulty adjustment system for players replay ability. In *2020 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)*, pages 860–866. IEEE, 2020.
- [23] Business Research Insights. Language learning application market size, share, growth, and industry analysis. <https://www.businessresearchinsights.com/marke>

- t-reports/language-learning-application-market-102456, 2026. Accessed: 2026-07-28.
- [24] Athanasios Karasimos. The battle of language learning apps: a cross-platform overview. *Research Papers in Language Teaching and Learning*, 12(1):150–166, 2022.
- [25] Duolingo, Inc. shareholder letter q2 2026, 2026.
- [26] Duolingo, Inc. Duolingo launches 148 new language courses. Press Release, Duolingo Investor Relations, <https://investors.duolingo.com/news-releases/news-release-details/duolingo-launches-148-new-language-courses>, April 2025. Accessed: 2026-07-28.
- [27] Alex Gilmore. Authentic materials and authenticity in foreign language learning. *Language Teaching*, 40(2):97–118, 2007.
- [28] Hui Liu and Hu Dong. Authentic language models and authenticity in the foreign language classrooms. *Journal of Educational Research and Policies*, 6:61–66, 08 2024.
- [29] Sebastian Deterding, Dan Dixon, Rilla Khaled, and Lennart Nacke. From game design elements to gamefulness: Defining gamification. In *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, pages 9–15. ACM, 2011.
- [30] Jan L. Plass, Bruce D. Homer, and Charles K. Kinzer. Foundations of game-based learning. *Educational Psychologist*, 50(4):258–283, 2015.
- [31] Lev S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA, 1978. Edited by Michael Cole, Vera John-Steiner, Sylvia Scribner, and Ellen Souberman.
- [32] James P. Lantolf, editor. *Sociocultural Theory and Second Language Learning*. Oxford University Press, Oxford, 2000.
- [33] HelloTalk Inc. HelloTalk – learn languages with native speakers. <https://www.hellotalk.com>, 2026. Accessed: 2026-09-15.
- [34] Stephen D. Krashen. *The Input Hypothesis: Issues and Implications*. Longman, London, 1985.

- [35] Marcella Hu and I.S.P. Nation. Unknown vocabulary density and reading comprehension. *Reading in a Foreign Language*, 13(1):403–430, 2000.
- [36] I.S.P. Nation. *Learning Vocabulary in Another Language*. Cambridge University Press, Cambridge, UK, 2001.
- [37] George A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97, 1956.
- [38] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
- [39] Sebastian Leitner. *So lernt man lernen: Der Weg zum Erfolg*. Herder, Freiburg, 1972.
- [40] Burr Settles and Brendan Meeder. A trainable spaced repetition model for language learning. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1848–1858, 2016.
- [41] Junyao Ye, Jingyong Su, and Yilong Cao. A stochastic shortest path algorithm for optimizing spaced repetition scheduling, 08 2022.
- [42] Open Spaced Repetition. The algorithm. GitHub wiki, <https://github.com/open-spaced-repetition/awesome-fsrs/wiki/The-Algorithm>, 2026. Accessed: 2026-09-16.
- [43] Henry L. Roediger and Jeffrey D. Karpicke. Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, 17(3):249–255, 2006.
- [44] Robert Vanderplank. The value of teletext sub-titles in language learning. *ELT Journal*, 42(4):272–281, 1988.
- [45] Elke Peters and Stuart Webb. Incidental vocabulary acquisition through viewing L2 television and factors that affect learning. *Studies in Second Language Acquisition*, 40(3):551–577, 2018.
- [46] Maribel Montero Perez, Elke Peters, and Piet Desmet. Vocabulary learning through viewing video: The effect of two enhancement techniques. *Computer Assisted Language Learning*, 31(1-2):1–26, 2018.

- [47] Robert Godwin-Jones. Games in language learning: Opportunities and challenges. *Language Learning & Technology*, 18(2):9–19, 2014.
- [48] Julie M. Sykes and Jonathon Reinhardt. *Language at Play: Digital Games in Second and Foreign Language Teaching and Learning*. Pearson Education, New York, 2013.
- [49] Gediminas Adomavicius and Alexander Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749, 2005.
- [50] Robin Burke. Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4):331–370, 2002.
- [51] Francesco Ricci, Lior Rokach, and Bracha Shapira. Introduction to recommender systems handbook. In Francesco Ricci, Lior Rokach, Bracha Shapira, and Paul B. Kantor, editors, *Recommender Systems Handbook*, pages 1–35. Springer, 2011.
- [52] Aleksandra Klačnjak-Milićević, Boban Vesin, Mirjana Ivanović, and Zoran Budimac. E-learning personalization based on hybrid recommendation strategy and learning style identification. *Computers & Education*, 56(3):885–899, 2011.

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CALL	Computer-Assisted Language Learning
CEFR	Common European Framework of Reference for Languages
CF	Collaborative Filtering
CLT	Cognitive Load Theory
CPU	Central Processing Unit
DDA	Dynamic Difficulty Adjustment
DHP	Difficulty, Half-life, and Probability of recall
DSR	Difficulty, Stability, and Retrievability
FR_{<i>i</i>}	Functional Requirement number <i>i</i> (FR1–FR7, Table C.1)
FSRS	Free Spaced Repetition Scheduler
HLR	Half-Life Regression
HS256	HMAC using SHA-256
HTTP	Hypertext Transfer Protocol
ISO	International Organization for Standardization
JIT	Just-In-Time (compilation)
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
L2	Second Language
LLM	Large Language Model
LLP	Language Learning Platform
MALL	Mobile-Assisted Language Learning
NFR_{<i>i</i>}	Non-Functional Requirement number <i>i</i> (NFR1–NFR7, Table C.2)
NLP	Natural Language Processing
OAuth	Open Authorization
ORM	Object-Relational Mapping
POS	Part of Speech

REST	Representational State Transfer
RQ	Research Question
RTL	Right-to-Left
S3	Simple Storage Service
SDK	Software Development Kit
SLA	Second Language Acquisition
SQL	Structured Query Language
SRS	Spaced Repetition System
TF-IDF	Term Frequency–Inverse Document Frequency
UI	User Interface
URL	Uniform Resource Locator
ZPD	Zone of Proximal Development

Glossary

Active Recall	Retrieving a target item from memory unaided, for example recalling and typing a word, rather than recognising it among distractors; the review format favoured by the testing effect
Algorithm	A sequence of instructions for solving a problem
Authentic Input	Language produced by native speakers for native speakers within a genuine communicative context, rather than written or graded for learners
Auto-Seeding	At registration, inserting every dictionary word below the learner's declared CEFR level into their tracked vocabulary, directly in FSRS's Review state with a randomised initial stability
Bag of Words	A representation of a text as the unordered counts of the words in it, discarding word order; <i>Glosy</i> represents each reel as a bag of word ids
Cascade Hybrid Recommender	A hybrid recommender in which one technique coarsely ranks or partitions the candidates and a second is applied only to break ties within that ranking, per Burke; the strategy of <i>Glosy</i> 's three-stage engine
CEFR Level	One of the six proficiency levels of the Common European Framework of Reference for Languages, from A1 (beginner) to C2 (mastery); used to tag dictionary words and to record a learner's declared level
Cognitive Load Theory	The theory that working memory is strictly limited, so instruction should minimise extraneous load (imposed by poor presentation) to leave capacity for germane load (the work of building long-term schemata)
Cold-Start Problem	The difficulty a recommender faces when it has few users or little accumulated interaction history, which particularly affects collaborative filtering
Collaborative Filtering	Recommending items from patterns of agreement across many users' behaviour, rather than from an item's own features; not used in <i>Glosy</i>
Comprehensibility Filter	Stage 1 of <i>Glosy</i> 's recommender: a hard pass/fail filter that removes a reel if its lexical coverage of the learner's tracked vocabulary is below 98%, or if it was recommended to that learner within the cooldown window

Comprehensible Input	Language input a learner can understand despite containing some unknown elements, per Krashen's Input Hypothesis ($i + 1$)
Content-Based Filtering	Recommending items by comparing an item's own features with a profile of one user's preferences built from that user's history, without reference to other users
Cosine Similarity	A measure of the similarity of two vectors, given by the cosine of the angle between them; <i>Glosy</i> ranks reels by their cosine similarity to the learner's profile vector
Diacritics	Marks added to letters that modify their pronunciation or meaning, such as Greek accents and Farsi harakat (short-vowel marks); stripped from words before comparison in the Wordle-style game
Dialogue	In <i>Glosy</i> 's data model, the ordered sequence of timestamped subtitle sentences that forms a reel's transcript
DSR Model	The memory model underlying FSRS, which represents an item's state by three jointly updated variables: difficulty (how intrinsically hard the item is), stability (the days for retrievability to decay to 90%), and retrievability (the momentary probability of recall)
Due Word	A tracked word whose scheduled review time has been reached; the frontend supplies these to the recommendation engine oldest-first, and the games draw their practice word from them
Dynamic Difficulty Adjustment	Algorithmically tuning a game's difficulty to the individual player's observed performance rather than fixing it in advance
Engagement Signal	A learner-generated record of interaction with a piece of content, such as a view, like, save, share, or comment; weighted by strength, these form the learner's profile vector
Forgetting Curve	The exponential decline of memory retention over time in the absence of reinforcement, first described by Ebbinghaus
Free Spaced Repetition Scheduler	An open-source spaced-repetition algorithm, descended from the DHP model, that updates each item's DSR state after every graded review; <i>Glosy</i> runs it on-device with the published default parameters
Game-Based Learning	Learning in which the instructional content is embedded in the core mechanical loop of a game, so that linguistic competence is needed to win rather than merely rewarded afterwards; contrasted with gamification
Gamification	The addition of game-design elements (points, streaks, badges) to a non-game task, as distinct from game-based learning
Half-Life Regression	Duolingo's spaced-repetition model (Settles and Meeder), which estimates a per-item, per-learner memory half-life by regression over a large corpus of pooled review data
Hypercasual Game	A mobile-native game with a minimalist interface, instant tap-to-play mechanics, and a very short core loop that needs no learning curve; hybrid-casual games add mid-core features such as progression and item collection

Incidental Vocabulary Acquisition	Learning words as a by-product of exposure, such as watching captioned video, without an explicit study task
JSON Web Token (JWT)	A compact, signed token used to authenticate API requests; <i>Glosy</i> issues a short-lived access token and a longer-lived refresh token, and both backend services verify the same access token
Language Learning Platform	A digital application or service for studying a language, such as Duolingo, Memrise, Busuu, Babbel, or LingQ
Leitner System	A non-parametric flashcard scheduling method in which a correctly answered card moves to a box reviewed at a longer interval, and an incorrectly answered card returns to the first box
Lemmatisation	Reducing an inflected word form to its dictionary base form (lemma), so that different forms of a word map to one vocabulary item; performed offline in <i>Glosy</i> by a backfill script
Letter Budget	The cap of 8 on the shared pool of letter tiles in a Word of Wonders level: a word is placed only if the multiset union of its letters with those already in use (the per-letter maximum count, not the sum) stays within it
Lexical Coverage	The proportion of word tokens in a text that a reader already knows; 98% coverage is the established threshold for unassisted comprehension
Microlearning	Decomposing instruction into short, self-contained units so that a single episode cannot exceed working-memory capacity; the basis of <i>Glosy</i> 's 90-second reel cap and its bounded game rounds
Mobile-Assisted Language Learning Offline-First	Language learning supported by mobile devices, typically through self-directed applications available on demand A design in which the client keeps its own persistent copy of state, works without a connection, and reconciles its changes with the server when connectivity returns
Practice Word	The word around which a Word of Wonders board is built; it is graded as an FSRS review only when it was drawn from the due-word queue
Profile Vector	A learner's preference profile in Stage 3 of the recommender: the weighted sum of the TF-IDF vectors of the reels they have engaged with, each weighted by the strength of that engagement
Progression Currencies	Coins, energy, and experience: the light extrinsic-motivation layer of <i>Glosy</i> , of which only coins currently have a complete reward loop
Recommender System	An algorithm that ranks or filters candidate items for a user based on content features, collaborative behaviour, or both
Reel	In <i>Glosy</i> , a short-form video of at most 90 seconds with an attached dialogue of timestamped, tokenised subtitles from which its vocabulary is derived
Refresh Token Rotation	Issuing a new refresh token at every use and revoking the old one; <i>Glosy</i> groups tokens into families, so replaying an already-rotated token revokes the whole family

Review Grade	The learner's self-rated recall outcome for a review (again, hard, good, or easy), which FSRS uses to update a word's state; an <i>again</i> grade records a lapse
Second Language Acquisition	The process by which people learn a language other than their first, and the research field that studies it
Shallow Gamification	Gamification reduced to repetitive, decontextualised drills wrapped in points, streaks, and leaderboards, so that the reward layer is decorative rather than structural
Short-Form Video	Brief video native to social platforms, typically 15 seconds to 3 minutes long and optimised for rapid, algorithmically driven consumption
Sociocultural Theory	Vygotsky's account of learning as a socially mediated process in which language is the primary cognitive tool; the source of the zone of proximal development and of scaffolding, the temporary support given by a more capable other
Spaced Repetition	A learning technique that schedules review of an item at increasing intervals timed to precede predicted memory decay
Testing Effect	The finding that actively retrieving information from memory produces better long-term retention than passive re-study
TF-IDF	Term frequency–inverse document frequency: a weighting that scales down a term's frequency in a document by how common the term is across the whole collection; applied by <i>Glosy</i> to reels' word-id bags
Tokenisation	Splitting text into individual units (tokens), here words, each linked to a dictionary word id
Tracked Vocabulary	The set of words in a learner's <code>user_vocabulary</code> , each with its FSRS memory state; the system's model of what that learner knows
Word of Wonders	<i>Glosy</i> 's crossword-style game: a procedurally generated 8×8 grid of intersecting words drawn from the learner's tracked vocabulary, solved by picking letters from a shared pool
Wordle-Style Game	<i>Glosy</i> 's second game: six attempts to guess a five-letter word from the learner's tracked vocabulary, with each guess coloured letter by letter, in English, Greek, or Farsi
Zone of Proximal Development	The gap between what a learner can do unaided and what they can do with appropriate scaffolding, per Vygotsky's sociocultural theory

